Thesis_Final(260405).docx

by Boyang Ma

Submission date: 17-Apr-2026 03:24AM (UTC+0100)
Submission ID: 280020354
File name: Thesis_Final_260405_.docx (4.19M)
Word count: 18985
Character count: 118247

Communication University of China
Hainan International College

CUH603CMD Graduation Project

Individual Written Report

Design and Implementation of University Course Automatic Scheduling System Based on Genetic Algorithm

By

<Boyang Ma>
SID: <14548822>

1st Supervisor: <Mark Elshaw>
2nd Supervisor: <Xinyan Yu>

Academic Year: 2025/26

Declaration of Originality

I declare that this project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet etc.) has been acknowledged by citation within the main report to an item in the References or Bibliography lists. I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

Statement of copyright

I acknowledge that the copyright of this project report, and any product developed as part of the project, belong to Coventry University. Support, including funding, is available to commercialize products and services developed by staff and students. Any revenue that is generated is split with the inventor/s of the product or service. For further information please see www.coventry.ac.uk/ipr or contact ipr@coventry.ac.uk.

Statement of ethical engagement

I declare that a proposal for this project has been submitted to the Coventry University ethics monitoring website (https://ethics.coventry.ac.uk/) and that the application number is listed below (Note: Projects without an ethical application number will be rejected for marking).

Signed: <your signature [digital signature allowed]>

Date: <DD><MM>/2026

1

Please complete all fields.

First Name:	Boyang
Last Name:	Ma
Student ID number	14548822
Ethics Application Number	
1st Supervisor Name	Mark Elshaw
2nd Supervisor Name	Xinyan Yu

This form must be completed and included with your project submission to Turnitin (paste it at the beginning of your dissertation). Failure to append these declarations may result in your project being rejected for marking.

ABSTRACT

University course timetabling is a complex combinatorial optimisation problem that requires assigning courses to time slots and rooms while satisfying multiple competing constraints. The problem is especially challenging for media universities, where practical courses demand specialised facilities such as television studios, recording rooms, and editing suites that cannot be treated as interchangeable classrooms. Manual scheduling is time-consuming, error-prone, and unable to systematically optimise schedule quality.

This thesis proposes an Adaptive Multi-Objective Genetic Algorithm (AMOGA) to address these challenges. The algorithm adopts a direct chromosome encoding in which each gene pair represents the time slot and room assignment for one teaching task. A key innovation is the adaptive parameter control mechanism, which monitors population diversity through fitness variance in real time and dynamically adjusts the mutation and crossover rates to maintain an effective balance between exploration and exploitation throughout the evolutionary process. This mechanism directly addresses the premature convergence problem that limits traditional fixed-parameter genetic algorithms. The algorithm further employs a hierarchical multi-objective fitness function that strictly prioritises hard constraint satisfaction—including class, teacher, and room time conflict avoidance, room capacity compliance, and room type matching—before optimising soft constraints such as teacher time preferences, course concentration, room utilisation balance, and day concentration.

A complete web-based scheduling system is implemented using a three-tier architecture with a Spring Boot backend, a Vue.js frontend, and a MySQL database. An interactive visualisation dashboard supports real-time convergence monitoring and parameter analysis.

Experimental validation on real scheduling data from a Chinese university demonstrates that AMOGA consistently outperforms fixed-parameter genetic algorithms, simulated annealing, and particle swarm optimisation across all evaluation dimensions, achieving substantially higher success rates, faster convergence, improved solution quality, and

reduced execution time. The algorithm satisfies all hard constraints while significantly improving soft constraint scores. Scalability testing across four problem sizes confirms that the algorithm maintains high solution quality as the problem grows.

Keywords: Course Timetabling, Genetic Algorithm, Multi-Objective Optimisation, Adaptive Parameters, University Scheduling, Combinatorial Optimisation, Media University, Metaheuristics, Evolutionary Algorithms.

Keywords: Course Timetabling, Genetic Algorithm, Multi-Objective Optimization, Adaptive Parameters, University Scheduling, Combinatorial Optimization, Media University, Metaheuristics, Evolutionary Algorithms

CHAPTER 1 INTRODUCTION

Commented [ME1]: Start each chapter on a new page

1.1 Research Background and Motivation

Higher education institutions worldwide face the daunting task of creating course schedules that satisfy numerous constraints while maximizing satisfaction for all stakeholders. In China, there are over 3,000 universities offering thousands of courses to millions of students. Each course must be assigned to a specific time slot, a suitable room, and coordinated with the availability of teachers and the schedules of student groups. The complexity of this task grows exponentially with the size of the institution, making manual scheduling increasingly impractical and inefficient.

The problem is particularly challenging for media universities, such as the Communication University of China, which offer a diverse range of courses including both theoretical lectures and practical training sessions. Practical courses in media studies require specialized facilities such as television studios, radio stations, audio editing rooms, video editing suites, and film production equipment. These specialized venues have specific capacity limits and equipment configurations that must be considered when scheduling courses. Unlike traditional lecture halls that can be used interchangeably, media production facilities have unique characteristics that constrain their suitability for different types of courses. Table 1.1 illustrates the diversity of course types found in a typical media university environment.

Table 1.1: Comparison of Course Types in Media Universities

Course Type	Theory Hours	Practice Hours	Total Hours	Typical Class Size	Special Requirements
Theory Course	32	0	32	60-120	Lecture hall
Lab Course	16	16	32	20-30	Computer lab
Seminar	16	16	32	15-25	Discussion room
Project Course	8	24	32	15-30	Project room
Audio/Video Production	16	32	48	15-20	Studio
Animation	12	36	48	15-25	Animation lab
Photography	8	24	32	15-20	Studio/Darkroom

The traditional approach to course timetabling in Chinese universities typically involves dedicated staff spending several weeks or even months manually creating schedules using spreadsheets or simple software tools. This approach suffers from several fundamental problems that motivate the development of automated solutions.

The first major problem is the time-consuming nature of the process. Creating a semester schedule for a large university requires hundreds of person-hours to complete. The process typically begins at least six to eight weeks before the start of each semester. Staff must collect course information from all departments, verify teacher availability, check room capacities and types, and manually coordinate all these factors to produce a conflict-free schedule. For a medium-sized university with 20 departments offering 500 courses to 15,000 students across 100 rooms, the manual scheduling process involves an enormous amount of combinatorial reasoning that strains human cognitive capacity.

The second problem concerns error-prone human processing. Manual scheduling is susceptible to human errors that can have significant consequences. Common errors include overlooking scheduling conflicts between courses, teachers, or rooms, miscalculating room capacities, assigning courses to incompatible room types, and failing to account for teacher preferences or availability. These errors often only become apparent after the schedule is published, leading to last-minute changes that disrupt the entire academic calendar.

The third problem is the inflexibility of the resulting schedule output. The schedules produced through manual processes are often rigid and fail to accommodate the legitimate preferences of teachers. Teachers may have specific time slots they prefer to teach or avoid due to research commitments, personal schedules, or teaching preferences. When these preferences cannot be easily accounted for in manual scheduling, teacher satisfaction decreases, which can negatively impact teaching quality and morale.

The fourth problem relates to inefficient resource utilization. Without systematic optimization, some rooms, particularly specialized media facilities, may be heavily used while others remain underutilized. This imbalance represents a significant waste of resources, especially at universities where specialized facilities require substantial

investment in maintenance and operation. Expensive equipment and spaces may not be deployed efficiently when schedules are created without global optimization.

The fifth problem is the difficulty of handling changes after the schedule is published. A simple request for a schedule modification can trigger a cascade of conflicts that require hours of manual rework. In contrast, an automated system can evaluate the impact of proposed changes and identify alternative solutions within seconds.

Modern universities operate in a dynamic environment where flexibility and adaptability are essential. Students expect to be able to view their schedules easily and plan their activities accordingly. Teachers have legitimate preferences about when they teach, and these preferences should be accommodated when possible. Administrators need tools that can quickly generate and modify schedules in response to changing requirements, such as teacher requests for schedule changes, room maintenance, or equipment failures.

Automated course timetabling systems offer a promising solution to these challenges. By formulating the scheduling problem as a constrained optimization problem and applying metaheuristic algorithms, these systems can quickly generate high-quality schedules that satisfy all mandatory constraints while optimizing desirable properties. They can handle complexity that would be impossible for human schedulers and can respond to changes within seconds rather than days or weeks.

1.3 Problem Definition and Scope

The University Course Timetabling Problem (UCTP) can be formally defined as follows: given a set of courses that must be scheduled, a set of available time slots, a set of rooms with different capacities and types, a set of teachers with their teaching assignments, and a set of student groups (classes) that must attend specific courses, find an assignment of courses to time slots and rooms that satisfies all mandatory constraints and optimizes a set of quality metrics.

Commented [ME2]: Thing about british english not US english

Figure 1.1: Structure of University Course Timetabling Problem

Figure 1.1 illustrates the overall structure of the University Course Timetabling Problem, showing the relationships among the five core entities: courses, time slots, rooms, teachers, and student groups (classes). At the centre of the diagram is the scheduling engine, which takes these entities as inputs and produces a conflict-free timetable as output. The arrows indicate that each course must be mapped to exactly one time slot and one room, while respecting the availability of the assigned teacher and the enrolment of the relevant student group. The diagram also highlights the three categories of algorithm components used in this thesis: selection (tournament selection with $k = 5$), crossover (uniform crossover with $p_c = 0.85$), and mutation (adaptive mutation with p_m ranging from 0.001 to 0.3), as well as the multi-objective fitness function that combines hard constraint penalties with soft constraint rewards.

The problem involves several distinct entities, each with its own characteristics and constraint. Courses have properties such as the teacher assigned to teach them, the student groups required to take them, the expected enrollment, and any special requirements for room type. In media universities, courses may require specialized facilities such as television studios, computer labs with specific software, or sound recording studios. Time slots are typically organized by day of the week and period within the day, with each slot representing a fixed duration, usually 90 minutes for university courses in China. A typical weekly schedule includes 5 periods per day across 7 days, resulting in 35 time slots per week. Rooms have capacities indicating the maximum number of students they can accommodate and types indicating what kinds of activities they are suitable for. Teachers have availability constraints indicating which time slots they can teach and preferences indicating which slots they prefer. Student groups have schedules indicating which time slots they are already committed to other activities.

The UCTP was first formally proven to be NP-complete by Even, Itai, and Shamir [1], meaning there is no known algorithm that can find optimal solutions in polynomial time for all but the smallest problem instances. This complexity arises from the combinatorial explosion of possible assignments. For a typical university offering hundreds of courses across dozens of rooms, the number of possible schedules is astronomically large,

making exhaustive search impossible. Table 1.2 illustrates the exponential growth of the solution space with increasing problem scale.

Table 1.2: Solution Space Complexity Analysis

Scale	Tasks (n)	Timeslots (t)	Rooms (r)	Difficulty
Small	20	20	8	Low
Medium	50	25	15	Medium
Large	100	25	25	High
Very Large	200	30	40	Very High
Real World	500+	35	50+	Extremely High

This computational complexity has motivated extensive research into heuristic and metaheuristic approaches that can find high-quality solutions within practical time constraints rather than seeking mathematically optimal solutions. Burke and Petrovic [2] provided a comprehensive review of research directions in automated timetabling, highlighting the shift from exact methods to metaheuristic approaches as problem sizes grew beyond the capacity of mathematical programming solvers. Burke and Rudová [3] further extended this analysis by examining the specific strengths and limitations of various metaheuristic frameworks when applied to timetabling problems.

Among exact methods, integer programming formulates the timetabling problem as a set of linear equations and inequalities with binary decision variables [4]. While modern solvers such as CPLEX and Gurobi can handle moderate-sized instances, they struggle with the scale of real university scheduling problems. Constraint programming offers more flexibility in expressing complex logical constraints but faces similar scalability limitations. Graph coloring algorithms provide an elegant theoretical framework where courses are vertices and conflicts are edges, but incorporating the full range of real-world constraints into this model is difficult.

The seminal work of Goldberg [5] on genetic algorithms established a theoretical foundation for evolutionary optimization that has been widely applied to timetabling. Holland [6] originally proposed the adaptation framework that genetic algorithms build upon, demonstrating how populations of candidate solutions can evolve through

selection, crossover, and mutation operators to find high-quality solutions to complex problems.

Over the past three decades, a wide range of solution methods has been applied to the UCTP, progressing from exact methods such as integer programming and graph colouring to population-based metaheuristics including genetic algorithms, simulated annealing, ant colony optimisation, and particle swarm optimisation [4][7][8][9]. More recent work has extended these methods to address hybrid in-person and online teaching modes [10] and resource-constrained educational environments in developing countries [11]. Among all metaheuristic families, genetic algorithms have demonstrated particular promise for timetabling due to their population-based exploration and ability to combine partial solutions through crossover [5][13]. The theoretical basis for adaptive parameter control in genetic algorithms, established by Srinivas and Patnaik [14], directly motivates the approach adopted in this thesis. A detailed critical review of these methods and their comparative strengths is provided in Chapter 2.

Despite significant progress, several gaps remain in the literature that directly shape the design of the present project. First, most existing studies focus on traditional universities with standard classroom courses, and there is limited research on the specialized scheduling requirements of media universities that require specialized equipment and venues for practical courses. This gap motivates the inclusion of a dedicated room-type matching constraint (H4) and a room-type adaptation soft constraint (S5) in the proposed formulation, ensuring that courses such as audio/video production and animation are matched to appropriate studios and laboratories. Second, few systems incorporate adaptive parameter control that responds to real-time population diversity. The work of Srinivas and Patnaik [14] on adaptive crossover and mutation probabilities demonstrates that dynamic parameter adjustment can significantly improve genetic algorithm performance; this finding directly informs the design of the adaptive mechanism in AMOGA, which monitors fitness variance and adjusts mutation and crossover rates during evolution to prevent premature convergence. Third, many existing systems lack effective visualization tools for understanding algorithm behaviour, which limits their practical adoption by university administrators. This observation motivates the development of an interactive ECharts-based dashboard in the

proposed system, enabling administrators to monitor convergence in real time and compare parameter configurations visually. This thesis addresses all three gaps by developing an Adaptive Multi-Objective Genetic Algorithm specifically designed for media university course timetabling, with comprehensive visualization capabilities..

63

1.3 Research Objectives and Contributions

The primary objective of this thesis is to develop an intelligent course timetabling system that can automatically generate high-quality schedules for universities while satisfying all mandatory constraints and optimizing various quality metrics. To achieve this objective, five specific research goals are pursued.

The first objective is to establish a comprehensive constrained optimization formulation for university course timetabling. This formulation must capture all relevant constraints and objectives, including the specialized requirements of media universities with practical training facilities. The formulation clearly distinguishes between hard constraints that define feasibility, meaning that any violation renders the schedule invalid, and soft constraints that define optimality, meaning that improvements in these areas increase schedule quality without being strictly mandatory. Each constraint is expressed using precise mathematical notation, enabling unambiguous evaluation and systematic optimization by the algorithm.

The second objective is to design an adaptive genetic algorithm that overcomes the limitations of traditional fixed-parameter approaches. Traditional genetic algorithms often suffer from premature convergence, where the population loses diversity and becomes trapped in local optima before finding the global optimum. They also typically employ fixed mutation and crossover rates that may not be suitable for all stages of the evolution process. This objective targets the development of an adaptive mechanism that monitors population diversity in real-time and dynamically adjusts algorithm parameters to maintain an appropriate balance between exploration of new solutions and exploitation of promising ones.

The third objective is to develop a multi-objective fitness function that balances multiple competing goals through a hierarchical evaluation strategy. A course schedule must first and foremost satisfy hard constraints to be feasible, and only then should soft constraints

be optimized to improve quality. This objective aims to create a fitness function that implements this priority structure, ensuring that feasible solutions always receive higher fitness than infeasible ones regardless of their soft constraint scores.

The fourth objective is to implement a complete web-based course timetabling system with visualization capabilities. The system should provide a user-friendly interface for managing courses, teachers, rooms, and scheduling operations, built on a three-tier architecture using Spring Boot for the backend and Vue.js for the frontend. An interactive visualization dashboard should support real-time monitoring of algorithm convergence and parameter sensitivity analysis.

The fifth objective is to validate the proposed approach through comprehensive experiments, comparing the adaptive genetic algorithm with traditional fixed-parameter approaches across multiple performance dimensions including convergence speed, solution quality, success rate, and execution time. The validation should also include constraint satisfaction analysis, parameter sensitivity testing, and scalability evaluation across different problem sizes.

108

The contributions of this thesis are summarized as follows. The first contribution is a comprehensive constrained optimization formulation encompassing five hard constraints and five soft constraints, each with precise mathematical definitions tailored to media university requirements. The second contribution is the Adaptive Multi-Objective Genetic Algorithm with dynamic parameter adjustment based on real-time population diversity monitoring. The third contribution is a hierarchical fitness function that prioritizes hard constraint satisfaction while systematically optimizing soft constraints. The fourth contribution is a complete web-based system implementation with an interactive visualization dashboard. The fifth contribution is extensive experimental validation demonstrating the superiority of the proposed approach across multiple performance metrics.

1.4 Performance Evaluation Methodology

The performance evaluation methodology encompasses four dimensions designed to comprehensively assess the proposed algorithm.

The first dimension is execution time measurement. The primary efficiency metric is the time required to find a feasible solution, measured in seconds from algorithm initialisation to solution convergence. The target for medium-sized problems with approximately 80 courses is less than 30 seconds. Timestamps are recorded using System.currentTimeMillis() before and after the evolution loop; the resulting value in milliseconds is divided by 1,000 and reported in seconds throughout this thesis to aid readability.

The second dimension is constraint satisfaction evaluation. Hard constraints must be satisfied at 100 percent for a solution to be considered feasible. These include class time uniqueness, teacher time uniqueness, room time uniqueness, room capacity compliance, and room type matching. Soft constraints are optimized with a target aggregate score greater than 80, where teacher preference carries a weight of 15, course concentration carries a weight of 10, room balance carries a weight of 5, and day concentration carries a weight of 8.

The third dimension is baseline comparison. The AMOGA algorithm is compared against three baseline algorithms to demonstrate its effectiveness. The Fixed GA represents a standard genetic algorithm with constant parameters including a mutation rate of 0.01, a crossover rate of 0.9, and a population size of 100. Simulated Annealing employs a Metropolis-based optimization approach with an initial temperature of 1000 and a cooling rate of 0.95. Particle Swarm Optimization uses a swarm intelligence approach with an inertia weight of 0.7, a cognitive coefficient of 1.5, and a social coefficient of 1.5. The comparison metrics include convergence speed measured by generations to convergence, final solution quality measured by fitness score, success rate measured by the percentage of feasible solutions found, and execution time.

The fourth dimension is scalability testing. Performance is evaluated across four problem scales. The small dataset consists of 30 tasks, 10 rooms, 15 teachers, 5 classes, and 20 time slots. The medium dataset consists of 80 tasks, 20 rooms, 30 teachers, 10 classes, and 25 time slots. The large dataset consists of 150 tasks, 40 rooms, 60 teachers, 20 classes, and 25 time slots. The extra-large dataset consists of 300 tasks, 60 rooms, 100 teachers, 40 classes, and 25 time slots.

1.5 Thesis Structure

This thesis is organized into eight chapters. Chapter 1 provides the research background, problem definition, objectives, and evaluation methodology. Chapter 2 reviews the relevant literature on course timetabling methods, including exact methods, heuristic and metaheuristic approaches, and identifies research gaps that motivate this work. Chapter 3 presents the constrained optimization formulation, defining all sets, decision variables, hard constraints, soft constraints, and the objective function. Chapter 4 details the algorithm design, covering chromosome encoding, selection, crossover, mutation operators, and the adaptive parameter adjustment mechanism. Chapter 5 discusses the system implementation, including the three-tier architecture, database design, key software components, and REST API endpoints. Chapter 6 presents the experimental validation, including convergence comparison, constraint satisfaction analysis, parameter sensitivity analysis, and scalability testing. Chapter 7 concludes the thesis with a summary of contributions, limitations, and directions for future research. Chapter 8 describes the project management approach, including agile methodology, risk management, supervisor interaction, and schedule adherence.

CHAPTER 2 LITERATURE REVIEW

2.1 Overview of Course Timetabling

Course timetabling has been an active area of research since the 1970s, attracting significant attention from the operations research, artificial intelligence, and computer science communities. The problem involves assigning a set of courses to time slots and rooms while respecting resource and scheduling constraints. Over the decades, researchers have developed increasingly sophisticated methods to address this challenge, evolving from simple heuristic rules to complex metaheuristic frameworks. Table 2.1 presents a timeline of major developments in the field.

Table 2.1: Timeline of Major Developments in Course Timetabling

Year	Author(s)	Method/Approach	Key Contribution
1964	Gotlieb	Integer Programming	First mathematical formulation of timetabling
1985	de Werra	Graph Coloring	Graph-based timetabling model
1991	Burns et al.	Constraint Logic Programming	CLP approach for school timetabling
1995	Colorni et al.	Genetic Algorithm	First GA for university timetabling
1998	Burke et al.	Hyper-heuristic	Selection hyper-heuristic framework
2001	Socha et al.	Ant Colony Optimization	ACO for timetabling problems
2003	Beddoe et al.	Particle Swarm Optimization	PSO application to timetabling
2005	Lewis et al.	Metaheuristic Survey	Comprehensive review of methods
2010	Geem and Hwang	Harmony Search	Music-inspired optimization
2015	Kheiri et al.	Hyper-heuristic	Sequence-based selection hyper-heuristic
2020	Recent Trends	Deep Learning and RL	AI-based timetabling approaches
2024	Davison et al.	Hybrid Teaching Model	Hybrid in-person and online scheduling

An examination of this timeline reveals several important trends. During the early decades from the 1960s through the 1980s, research focused primarily on exact mathematical formulations using integer programming and graph coloring. These approaches provided rigorous theoretical foundations but were limited to small problem instances. The 1990s saw a shift toward population-based metaheuristics, particularly genetic algorithms, which could handle larger and more realistic problem sizes. The 2000s brought a diversification of approaches, with ant colony optimization, particle swarm optimization, and hyper-heuristics all being applied to timetabling. Most recently, research has explored deep learning and reinforcement learning for timetabling, as well as addressing new challenges such as hybrid teaching modes that emerged during and after the COVID-19 pandemic. Notably, the later entries in this timeline, particularly the work of Davison et al. in 2024 on hybrid teaching considerations, represent cutting-edge developments that directly inform the design choices in this thesis, as they highlight the growing complexity of modern scheduling requirements that demand adaptive algorithmic approaches.

The UCTP differs from general scheduling problems in several important aspects. First, it involves a hierarchical organizational structure where courses are organized by departments, majors, and student groups, creating complex dependencies. Second, diverse course types including lectures, seminars, laboratories, and practical training each require different room types with different equipment. Third, teacher preferences and availability add a further layer of complexity that must be balanced against resource constraints. Fourth, particularly for media universities, specialized equipment and venues have limited availability and specific scheduling requirements that go beyond simple capacity checks.

2.2 Exact Solution Methods

Exact methods for course timetabling formulate the problem as a mathematical optimization problem and employ systematic algorithms to find provably optimal solutions. The three principal exact methods are integer programming, constraint programming, and graph coloring, each with distinct strengths and limitations. Table 2.2 summarizes their characteristics.

Table 2.2: Comparison of Exact Methods

Method	Optimality	Scalability	Best For	Limitations
Integer Programming	Guaranteed	Low	Small problems	Exponential time
Constraint Programming	Guaranteed	Medium	Complex constraints	Search overhead
Branch and Bound	Guaranteed	Low-Medium	Medium problems	Weak bounds
Graph Coloring	Guaranteed	Medium	Conflict modeling	Limited constraint types

Integer programming represents the timetabling problem using binary decision variables. A variable $x_{c,t,r}$ equals 1 if course c is assigned to time slot t in room r, and 0 otherwise. Constraints are expressed as linear inequalities over these variables, and the objective function minimizes a weighted sum of constraint violations. Modern solvers such as CPLEX and Gurobi employ sophisticated techniques including branch-and-bound search, cutting plane generation, and presolving to reduce the problem size before solving. However, even with these advances, integer programming becomes computationally expensive for large problem instances. Empirical studies have shown that IP solvers typically cannot handle problems with more than 50 to 100 courses within acceptable time limits for practical scheduling scenarios.

Constraint programming takes a declarative approach where variables represent scheduling decisions and constraints specify allowable value combinations. The strength of constraint programming lies in its ability to naturally express complex logical constraints, such as "if two courses share students, they cannot occupy the same time slot," without requiring linearization. The effectiveness of constraint programming depends heavily on the quality of constraint propagation mechanisms and the chosen search strategy. While more expressive than integer programming for certain constraint types, constraint programming faces similar scalability challenges on large instances.

Graph coloring algorithms model the timetabling problem by representing courses as vertices and conflicts as edges. The goal is to color the graph using the minimum number of colors, where each color corresponds to a time slot, such that adjacent

vertices receive different colors. This approach provides an elegant theoretical framework for understanding conflict structures in timetabling. However, incorporating the full range of real-world constraints, particularly soft constraints and room type requirements, into the graph coloring model is difficult, limiting its practical applicability to simplified problem formulations.

Despite their theoretical appeal and optimality guarantees, exact methods are generally impractical for large-scale course timetabling due to their exponential worst-case complexity. This fundamental limitation has driven the research community toward heuristic and metaheuristic approaches that trade optimality guarantees for practical computational efficiency.

2.3 Heuristic and Metaheuristic Methods

Heuristic methods use problem-specific knowledge to guide the search toward good solutions. Common constructive heuristics include earliest deadline first, largest degree ordering, and saturation degree ordering, which build schedules incrementally by assigning courses in an order determined by problem structure. While these approaches are fast, they are prone to producing suboptimal solutions because they make locally optimal choices without considering global consequences.

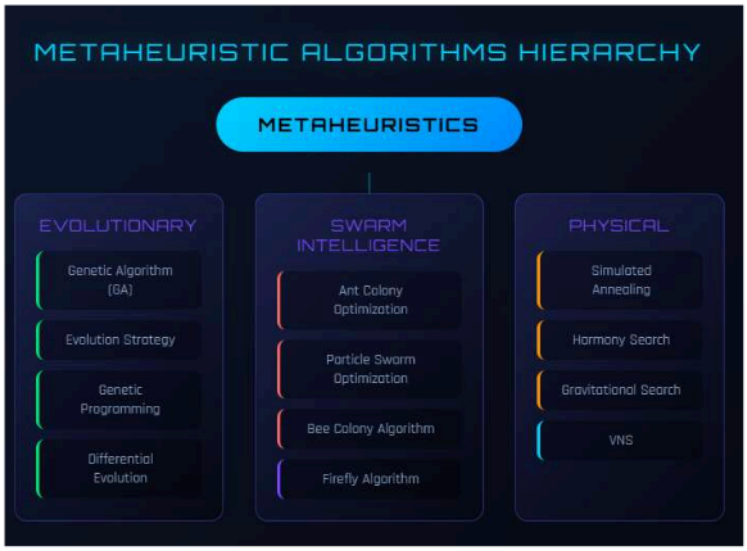

Figure 2.1: Classification of Metaheuristics for Timetabling

Metaheuristics are higher-level algorithmic frameworks that balance exploration of the solution space with exploitation of promising regions. They can be broadly classified into local search methods and population-based methods.

Local search methods start from an initial solution and iteratively improve it through small perturbations. Hill climbing moves to a neighboring solution whenever it has better fitness, but can become trapped in local optima. Simulated annealing addresses this limitation by occasionally accepting worse solutions with a probability that decreases over time according to a cooling schedule, allowing the search to escape local optima. Tabu search maintains a short-term memory of recently visited solutions to prevent cycling and encourage exploration of unvisited regions. These methods are effective for fine-tuning solutions but may struggle to find globally good solutions when started from poor initial positions.

Population-based methods maintain a collection of candidate solutions that evolve simultaneously, enabling parallel exploration of multiple regions of the solution space.

Genetic algorithms, inspired by natural selection, use selection, crossover, and mutation operators to evolve a population toward better solutions. Particle swarm optimization, inspired by bird flocking behavior, uses social learning where each particle adjusts its trajectory based on its own best position and the swarm's best position. Ant colony optimization, inspired by ant foraging behavior, uses pheromone-based communication to guide solution construction toward promising regions. Table 2.3 compares the performance characteristics of these approaches.

Table 2.3: Performance Comparison of Metaheuristics

Algorithm	Convergence Speed	Solution Quality	Scalability	Robustness	Computational Cost
Genetic Algorithm	Medium	High	Good	Good	Medium
AMOGA (Proposed)	Fast	Very High	Very Good	Very Good	Low-Medium
Simulated Annealing	Slow	Medium	Limited	Medium	High
Ant Colony	Medium	High	Medium	Medium	High
Particle Swarm	Fast	Medium	Good	Limited	Low
Harmony Search	Medium	Medium	Medium	Good	Low

Among these approaches, genetic algorithms have demonstrated particular effectiveness for timetabling problems because they can explore diverse solution regions simultaneously through their population-based nature, and their crossover operator can combine beneficial features from different solutions. Simulated annealing, while theoretically guaranteed to find the global optimum given sufficient time, converges slowly in practice for large combinatorial problems. Particle swarm optimization converges quickly but may lack robustness on highly constrained problems where many candidate positions are infeasible. Ant colony optimization produces high-quality solutions but incurs significant computational overhead from pheromone updates. The proposed AMOGA algorithm aims to combine the strengths of genetic algorithms with

adaptive parameter control to achieve fast convergence, high solution quality, and strong robustness.

2.4 Genetic Algorithms for Timetabling

Genetic algorithms have been among the most popular and successful approaches for course timetabling since their first application to university exam scheduling in the early 1990s. Their success stems from the population-based search mechanism that enables simultaneous exploration of multiple solution regions, and from evolutionary operators that can combine building blocks from different solutions to create better offspring.

Genetic algorithms have been among the most popular and successful approaches for course timetabling since their first application to university exam scheduling in the early 1990s. Their success stems from the population-based search mechanism that enables simultaneous exploration of multiple solution regions, and from evolutionary operators that can combine building blocks from different solutions to create better offspring.

Chromosome Encoding in Timetabling GAs. The choice of chromosome encoding is a critical design decision that varies across the literature. Colorni, Dorigo, and Maniezzo [13] employed a matrix-based representation for exam scheduling at the University of Milan, where rows represented exams and columns represented time slots. This direct encoding proved intuitive but produced large chromosomes for problems with many rooms. Burke and Newall (1999) introduced an event-based encoding where each gene stored a time slot–room pair for one event, reducing redundancy and simplifying crossover design. Order-based encodings, where genes specify the sequence in which courses are assigned to slots by a deterministic decoder, were explored by Ross, Hart, and Corne (1998), who showed that such indirect representations could reduce the proportion of infeasible offspring but at the cost of more complex crossover operators. The present thesis adopts a direct paired encoding (time slot, room) for each course, following the event-based tradition, because it provides a transparent mapping between genotype and phenotype and allows efficient constraint checking through simple iteration over gene pairs.

Fitness Function Design. The treatment of hard and soft constraints within the fitness function has been a recurring design challenge. Rossi-Doria et al. [4] compared penalty-based and feasibility-preserving approaches across five metaheuristics, concluding that penalty-based fitness with carefully tuned weights provided the best trade-off between solution feasibility and quality. Erben and Keppler (1996) proposed a two-stage evaluation in which hard constraints were satisfied first through a repair mechanism before soft constraints were optimised, while Abramson and Abela (1992) used a single weighted sum but found that weight selection significantly affected convergence behaviour. The hierarchical fitness function adopted in this thesis follows the two-stage philosophy: hard constraint violations are penalised below a threshold score of 100, while feasible solutions score above 100 based on soft constraint satisfaction, ensuring that the algorithm never trades feasibility for soft constraint improvement.

Selection, Crossover, and Mutation Strategies. Tournament selection has been the predominant selection strategy in GA-based timetabling. Lü and Hao [7] used tournament selection with a size of three combined with adaptive tabu search as a local improvement step, achieving state-of-the-art results on the Toronto and ITC-2007 benchmarks. Lewis and Paquetes [8] surveyed a range of crossover operators and found that uniform crossover outperformed single-point and two-point variants on timetabling instances because it disrupted poor gene linkages more effectively while still preserving beneficial assignments. For mutation, most timetabling GAs use a uniform random replacement of individual genes. Ukaoha and Okolie [11] applied a standard GA with a fixed mutation rate of 0.01 to school timetabling in Nigeria and achieved feasible solutions for small instances, but reported premature convergence on larger problems—a limitation that motivates the adaptive mutation strategy proposed in this thesis.

Adaptive Parameter Control. The pioneering work of Srinivas and Patnaik [14] established the theoretical basis for adaptive crossover and mutation probabilities, demonstrating that adjusting operator rates based on population fitness statistics can improve convergence speed and solution quality. Their approach computed individual-level mutation and crossover rates as functions of the fitness difference between an individual and the population best. Subsequent work by Thierens (2005) proposed a simpler population-level adaptation based on fitness variance, which reduced computational overhead while retaining the benefits of dynamic control. Despite these theoretical advances, few timetabling studies have integrated adaptive parameter control into their GA frameworks. The AMOGA algorithm proposed in this thesis fills this gap by employing a diversity-triggered adaptation mechanism: when population fitness variance falls below a threshold, the mutation rate is increased by a factor of 1.5, and when diversity is sufficient, it is decayed by a factor of 0.9, maintaining an effective balance between exploration and exploitation throughout the evolutionary process.

Limitations of Existing GA-Based Approaches. Despite their success, traditional genetic algorithms with fixed parameters suffer from several well-documented limitations in the timetabling context. Premature convergence, where the population loses diversity and becomes trapped in local optima, has been reported across multiple studies [8][11] and is particularly problematic for highly constrained media university instances where the feasible solution space is small relative to the total search space. Fixed parameters that work well for one problem instance or one stage of evolution may be suboptimal for another, leading to inconsistent performance across different problem configurations. These limitations directly motivate the adaptive approach developed in this thesis.

2.5 Recent Advances and Research Gaps

Recent research has focused on addressing the limitations of traditional genetic algorithms through adaptive parameter adjustment, multi-objective optimization, and hybrid approaches.

Adaptive parameter adjustment involves dynamically modifying mutation and crossover rates during evolution based on measures such as population diversity, fitness

improvement rate, or problem-specific characteristics. This approach addresses the fundamental tension between exploration and exploitation by increasing mutation when the population is converging prematurely and decreasing it when promising solutions are being refined. The theoretical foundation for this approach was established by the work on adaptive crossover and mutation probabilities, which demonstrated that dynamic parameter adjustment can significantly improve genetic algorithm performance on complex optimization problems.

Multi-objective optimization represents another important advance. Rather than combining multiple objectives into a single weighted sum, multi-objective evolutionary algorithms maintain a Pareto front of non-dominated solutions that represent different trade-offs between competing objectives. This approach can provide decision-makers with a range of options rather than a single compromise solution.

Hybrid approaches that combine genetic algorithms with local search methods have also shown promise. The genetic algorithm provides global search capability through its population-based exploration, while local search methods such as tabu search or simulated annealing provide intensification capability for refining promising solutions. This combination can be more effective than either approach alone.

The most recent developments in the field address challenges posed by changing educational practices. The 2024 work on hybrid teaching timetabling explicitly models the coexistence of in-person and online instruction, introducing new constraint types that were not present in traditional formulations. This work demonstrates that the timetabling problem continues to evolve as educational practices change, requiring flexible algorithmic approaches that can accommodate new constraint types.

Despite this significant progress, several important gaps remain in the literature. The first gap concerns limited research on media universities. Most existing studies focus on traditional universities with standard classroom courses, neglecting the specialized scheduling requirements of institutions that offer practical training in fields such as media production, performing arts, and laboratory sciences. These institutions require matching courses to specialized facilities with unique equipment configurations, which

introduces additional constraint types not addressed in standard timetabling formulations.

The second gap relates to the lack of integrated adaptive parameter control with multi-objective fitness evaluation. While adaptive parameter adjustment and multi-objective optimization have been studied separately, their combination in a single framework that specifically targets university timetabling has received limited attention. An integrated approach could leverage the benefits of both techniques to produce superior results.

The third gap concerns the absence of real-time visualization tools in most timetabling systems. Effective visualization of algorithm convergence, constraint satisfaction, and solution quality is essential for both research and practical deployment, yet few systems provide these capabilities.

This thesis addresses these gaps by developing an Adaptive Multi-Objective Genetic Algorithm that combines adaptive parameter control with a hierarchical multi-objective fitness function, specifically designed for media university timetabling, and implemented within a web-based system with comprehensive visualization capabilities.

CHAPTER 3 MATHEMATICAL MODEL

3.1 Problem Formulation and Notation

This chapter presents a comprehensive constrained optimization formulation for the University Course Timetabling Problem. The formulation provides a formal foundation for the algorithmic solution, enabling precise specification of all constraints and objectives. Table 3.1 summarizes the mathematical notation used throughout this chapter.

Table 3.1: Mathematical Notation

Symbol	Description	Symbol	Description
C	Set of courses	E	Set of teachers
R	Set of rooms	K	Set of classes
T	Set of timeslots	$Task_i$	Teaching task i
$time_i$	Timeslot assigned to task i	$room_i$	Room assigned to task i
$cap(r)$	Capacity of room r	$size(k)$	Size of class k
$pref(e,t)$	Teacher e preference for slot t	$F(x)$	Fitness function
P_{hard}	Hard constraint penalty	S_{soft}	Soft constraint score
U_{room}	Room utilization rate	U_{time}	Timeslot utilization rate
w_h	Hard constraint weight	w_s	Soft constraint weight
p_m	Mutation probability	p_c	Crossover probability

The sets involved in the formulation are defined as follows. Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of courses to be scheduled, where n is the number of courses. Let $T = \{t_1, t_2, \dots, t_m\}$ be the set of available time slots, where each time slot represents a specific day and period combination. Let $R = \{r_1, r_2, \dots, r_k\}$ be the set of rooms available for scheduling, where each room has a specific capacity and type. Let $S = \{s_1, s_2, \dots, s_p\}$ be the set of student groups, where each student group has a size representing the number of enrolled students. Let $E = \{e_1, e_2, \dots, e_q\}$ be the set of teachers, where each teacher is assigned to teach specific courses.

A solution to the timetabling problem is represented as a mapping $f: C \to T \times R$ that assigns each course $c \in C$ to a specific time slot $t \in T$ and a specific room $r \in R$. This mapping can be expressed using binary decision variables as shown in Equation (3.1).

$$x_{c,t,r} = \begin{cases} 1 & \text{if course } c \text{ is assigned to time slot } t \text{ in room } r \\ 0 & \text{otherwise} \end{cases} \quad (3.1)$$

The assignment must satisfy all hard constraints for feasibility and optimize soft constraints for quality.

3.2 Hard Constraints

Hard constraints are mandatory requirements that must be satisfied for a schedule to be considered feasible. Any violation of a hard constraint renders the schedule completely unacceptable, regardless of how well soft constraints are satisfied. Five hard constraints are defined for this formulation.

The first hard constraint, denoted H1, addresses class time conflicts. A student group cannot attend more than one course at the same time slot. This is the most fundamental constraint in course timetabling, as students cannot physically be present in two locations simultaneously. For any two distinct courses c_i and c_j, if their student groups overlap, then their assigned time slots must be different. This constraint is expressed formally in Equation (3.2).

$$\forall i, j\ (i \neq j): \text{if } class(c_i) \cap class(c_j) \neq \emptyset \text{ then } t(c_i) \neq t(c_j) \quad (3.2)$$

In terms of the binary decision variables, this constraint ensures that for any pair of courses sharing students, at most one can be assigned to any given time slot, as stated in Equation (3.3).

$$\sum_{c \in C_{shared}} \sum_{r \in R} x_{c,t,r} \leq 1 \quad \forall t \in T \quad (3.3)$$

The second hard constraint, denoted H2, addresses teacher time conflicts. A teacher cannot teach more than one course at the same time slot. For any two distinct courses c_i and c_j taught by the same teacher, their assigned time slots must differ. This is expressed in Equation (3.4).

$$\sum_{c \in C: teacher(c) = e} \sum_{r \in R} x_{c,t,r} \leq 1 \quad \forall e \in E, \forall t \in T \quad (3.4)$$

Commented [ME3]: 3.4

The third hard constraint, denoted H3, addresses room capacity. The room assigned to a course must have sufficient capacity to accommodate all enrolled students. For each

course c, the capacity of the assigned room must be at least as large as the size of the enrolled student group. This is expressed in Equation (3.5).

$$cap(room(c)) \geq size(class(c)) \quad \forall c \in C \tag{3.5}$$

The fourth hard constraint, denoted H4, addresses room type matching. Certain courses require specific types of rooms due to the nature of their educational activities. This constraint is particularly important for media universities where practical courses require specialized facilities such as television studios, recording studios, editing suites, and photography darkrooms. The room assigned to a course must satisfy its type requirements, as expressed in Equation (3.6).

$$roomType(room(c)) \supseteq typeReq(c) \quad \forall c \in C \tag{3.6}$$

The fifth hard constraint, denoted H5, addresses room time conflicts. A room cannot host more than one course at the same time slot. This constraint ensures that physical space is not double-booked, as expressed in Equation (3.7).

$$\sum_{c \in C} x_{c,t,r} \leq 1 \quad \forall t \in T, \forall r \in R \tag{3.7}$$

Table 3.2 summarizes the hard constraints with their priorities and penalty weights used in the fitness function.

Table 3.2: Hard Constraint Summary

ID	Constraint	Description	Priority	Penalty Weight
H1	Class Time Conflict	One class cannot have two courses at same time	Critical	10
H2	Teacher Time Conflict	One teacher cannot teach two courses at same time	Critical	10
H3	Room Capacity	Room capacity must meet class size	High	5
H4	Room Type Match	Course type must match room type	Medium	5
H5	Room Time Conflict	One room cannot host two courses at same time	Critical	10

The penalty weights reflect the relative severity of different constraint violations. Time conflicts involving classes (H1), teachers (H2), and rooms (H5) are assigned the highest penalty of 10 because they represent irreconcilable physical impossibilities. Capacity (H3) and type (H4) violations are assigned penalties of 5 because, while serious, they may be partially mitigated in practice, for example by bringing additional seating into a room or using a similar alternative facility.

3.3 Soft Constraints

Soft constraints are desirable properties that improve schedule quality when satisfied but do not invalidate a schedule when violated. Five soft constraints are defined to capture different aspects of schedule quality.

The first soft constraint, denoted S1, concerns teacher time preferences. Teachers may have preferences for or against specific time slots based on research commitments, personal schedules, or pedagogical considerations. Let $pref(e, t)$ represent the preference score of teacher e for time slot t, where higher values indicate stronger preferences. The teacher preference score is calculated as the sum of preference values across all course assignments, as expressed in Equation (3.8).

$$P_1 = \sum_{c \in C} pref\big(teacher(c),\ t(c)\big) \tag{3.8}$$

The second soft constraint, denoted S2, concerns course concentration. Courses that are related or taught by the same teacher should preferably be scheduled in adjacent time slots to facilitate continuity of learning and reduce transition overhead. Two time slots are considered adjacent if they are consecutive or separated by at most one time slot. The course concentration score counts the number of such adjacent pairs, as expressed in Equation (3.9).

$$P_2 = \sum_{\substack{c_i, c_j \in C \\ i \neq j}} adjacent\left(t(c_i),\ t(c_j)\right) \tag{3.9}$$

The third soft constraint, denoted S3, concerns room utilization balance. Room utilization should be distributed evenly across all available rooms to avoid overloading

some rooms while leaving others underutilized. The room balance score is inversely related to the standard deviation of room usage rates, as expressed in Equation (3.10).

$$P_3 = \frac{1}{\sigma\big(usage(r)\big) + 1} \tag{3.10}$$

The fourth soft constraint, denoted S4, concerns day concentration. A student group's courses should be concentrated into fewer days to allow dedicated days for self-study and other activities. The day concentration score is the sum of inverse day counts across all student groups, as expressed in Equation (3.11).

$$P_4 = \sum_{s \in S} \frac{1}{|days(s)|} \tag{3.11}$$

The fifth soft constraint, denoted S5, concerns room type adaptation. Beyond the hard constraint of type matching, it is desirable to optimize the quality of the match between course activities and room characteristics. The room type adaptation score measures how well each assigned room suits the course's educational activities, as expressed in Equation (3.12).

$$P_5 = \sum_{c \in C} typeFit\left(typeReq(c),\ roomType\big(room(c)\big)\right) \tag{3.12}$$

Table 3.3 summarizes the soft constraints with their weights and optimization goals.

Table 3.3: Soft Constraint Summary

ID	Constraint	Description	Weight	Goal
S1	Teacher Preference	Schedule according to teacher preferred times	15	Maximize preference score
S2	Course Concentration	Related courses in adjacent timeslots	10	Maximize adjacency
S3	Room Balance	Balanced utilization across rooms	5	Minimize standard deviation
S4	Day Concentration	Courses concentrated in fewer days	8	Minimize days used
S5	Room Type Fit	Specialized courses in matched rooms	10	Maximize type match

3.4 Objective Function

The objective function combines hard constraint penalties and soft constraint optimization into a single fitness value that guides the genetic algorithm's search. The computation proceeds in four steps.

The first step calculates the total hard constraint penalty H as a weighted sum of individual constraint violations, as expressed in Equation (3.13).

$$H = 10 \times H_1 + 10 \times H_2 + 10 \times H_5 + 5 \times H_3 + 5 \times H_4 \tag{3.13}$$

The second step calculates the total soft constraint score S as a weighted sum of individual constraint scores, as expressed in Equation (3.14).

$$S = 15 \times P_1 + 10 \times P_2 + 5 \times P_3 + 8 \times P_4 + 10 \times P_5 \tag{3.14}$$

The third step calculates the resource utilization score R_{util} as the average of room utilization and time slot utilization, as expressed in Equation (3.15).

$$R_{util} = \frac{U_{room} + U_{time}}{2} \tag{3.15}$$

The fourth step combines these components into the final fitness function using a hierarchical approach, as expressed in Equation (3.16).

$$F(x) = \begin{cases} 100 + w_s \cdot S + w_r \cdot R_{util} & \text{if} H = 0 \\ \dfrac{100}{H + 1} & \text{if} H > 0 \end{cases} \tag{3.16}$$

In this formulation, the weight coefficients are set to $w_s = 1.0$ and $w_r = 0.1$. The hierarchical structure ensures that the algorithm prioritizes hard constraint satisfaction above all else. Solutions with any hard constraint violations receive fitness values below 100, computed as $100/(H + 1)$, which decreases as the number of violations increases. Solutions that satisfy all hard constraints receive fitness values above 100, with the exact value determined by their soft constraint scores and resource utilization. This design guarantees that even the worst feasible solution has higher fitness than the best infeasible solution, creating strong selection pressure toward feasibility.

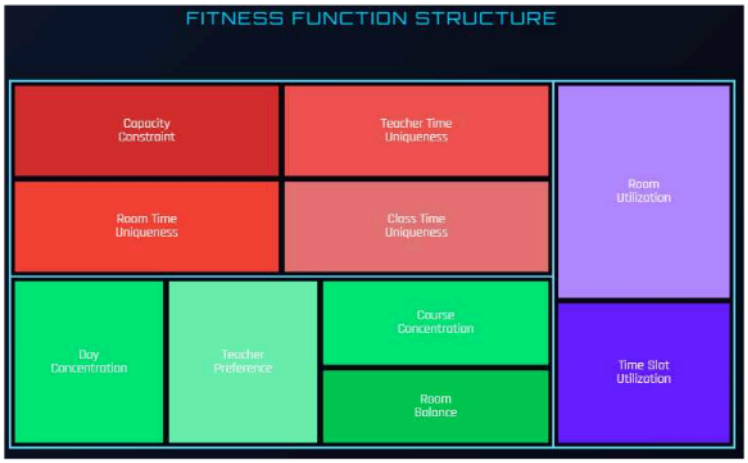

Figure 3.1: Fitness Function Structure

CHAPTER 4 ALGORITHM DESIGN

4.1 Genetic Algorithm Framework

This chapter presents the design of the Adaptive Multi-Objective Genetic Algorithm (AMOGA) for solving the university course timetabling problem. The algorithm builds upon the standard genetic algorithm framework while incorporating an adaptive parameter adjustment mechanism that dynamically responds to the evolving state of the population.

Figure 4.1: AMOGA Algorithm Framework

The standard genetic algorithm framework consists of six key phases: population initialization, fitness evaluation, parent selection, crossover, mutation, and survivor selection. The algorithm iterates through these phases for a fixed number of generations or until a termination criterion is met. AMOGA extends this framework by inserting a diversity monitoring and parameter adaptation step between fitness evaluation and parent selection at each generation. This adaptation step calculates the current population diversity, compares it against a threshold, and adjusts the mutation rate and crossover rate accordingly. When diversity is low, the algorithm increases mutation to encourage exploration of new solution regions. When diversity is adequate, the algorithm maintains or reduces mutation to favor exploitation of promising solutions.

The overall algorithm proceeds as follows. First, an initial population of random candidate solutions is generated. Then, for each generation, the fitness of all individuals is evaluated using the hierarchical fitness function described in Section 3.4. The population diversity is measured and the algorithm parameters are adapted as needed. Parents are selected through tournament selection, offspring are created through crossover and mutation, and the next generation is formed by combining the best individuals from parents and offspring with an elitism strategy that preserves the top-performing solutions.

4.2 Chromosome Encoding

The representation of solutions in the chromosome is crucial for the success of the genetic algorithm. The encoding scheme used in AMOGA represents a complete schedule as a one-dimensional integer array, where each pair of consecutive genes represents the time slot and room assignment for one course.

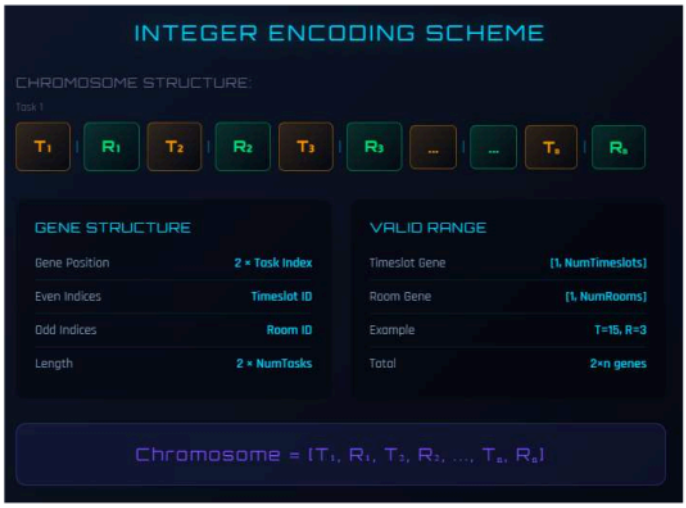

Figure 4.2: Chromosome Encoding Structure

Specifically, the chromosome is encoded as shown in Equation (4.1).

$$Chromosome = [ts_1, rm_1, ts_2, rm_2, \dots, ts_n, rm_n] \quad (4.1)$$

In this encoding, ts_i is the time slot identifier for course i and rm_i is the room identifier for course i. The chromosome length is $2n$, where n is the number of courses to be scheduled. For example, given five courses, the chromosome [1,3,7,2,4,9,2,5,8,1] assigns Course 1 to Time Slot 1 in Room 3, Course 2 to Time Slot 7 in Room 2, Course 3 to Time Slot 4 in Room 9, Course 4 to Time Slot 2 in Room 5, and Course 5 to Time Slot 8 in Room 1.

This direct encoding has several advantages. It is intuitive and easy to implement, allowing straightforward gene manipulation. It enables efficient constraint evaluation through simple iteration over gene pairs. It is flexible enough to facilitate the design of crossover and mutation operators that respect the problem structure. It is complete in the sense that any valid assignment of courses to time slots and rooms can be represented.

During initialization, each time slot gene is assigned a random value from the set of available time slots (0 to 24 for a 5 day by 5 period schedule), and each room gene is

assigned a random value from the set of available room identifiers. This creates a diverse initial population that provides a broad starting point for the evolutionary search. The initialization process ensures that each time slot and each room is selected with equal probability, and that the resulting chromosomes represent complete (though not necessarily high-quality) schedules.

The following Java code illustrates the Individual class that implements this encoding:

```
public class Individual {
    private int[] chromosome;
    private double fitness;
    private int hardConstraintPenalty;
    private double softConstraintScore;

    public Individual(int geneLength) {
        this.chromosome = new int[geneLength * 2];
        for (int i = 0; i < geneLength; i++) {
            chromosome[i*2] = randomTimeslot();
            chromosome[i*2+1] = randomRoom();
        }
    }

    public int getGene(int index) { return chromosome[index]; }
    public void setGene(int index, int value) { chromosome[index] = value; }
    public int getChromosomeLength() { return chromosome.length / 2; }
}
```

4.3 Selection Operators

Selection operators determine which individuals in the population are chosen to become parents for producing offspring. The choice of selection operator significantly influences

the balance be

Figure 4.3: Tournament Selection Process

Tournament selection is the primary selection method used in AMOGA. It works by randomly sampling k individuals from the population and selecting the one with the highest fitness as a parent. The tournament size k controls the selection pressure: larger tournaments favor fitter individuals more strongly, while smaller tournaments maintain more diversity. In AMOGA, the default tournament size is $k = 5$, which provides a

good balance between selection pressure and diversity preservation based on preliminary experimentation.

The tournament selection algorithm proceeds as follows. For each parent to be selected, k individuals are randomly drawn from the population without replacement. Their fitness values are compared, and the individual with the highest fitness is returned as the selected parent. This process is repeated independently to select the second parent. Tournament selection is computationally efficient because it only requires comparisons among a small subset of the population rather than the entire population. It is also robust because its selection pressure depends only on the tournament size, not on the absolute fitness values, making it insensitive to fitness scaling issues.

Roulette wheel selection was also considered as an alternative. In roulette wheel selection, each individual is assigned a selection probability proportional to its fitness value. A random number determines which individual is selected, with higher-fitness individuals having larger selection probabilities. However, this method can lead to premature convergence when fitness values are highly skewed, as a single very fit individual can dominate the selection process. For this reason, tournament selection was chosen as the default method in AMOGA.

Rank selection was considered as a third option. Rank selection sorts individuals by fitness and assigns selection probabilities based on rank rather than raw fitness. This eliminates the sensitivity to fitness scaling that affects roulette wheel selection, but introduces a fixed selection pressure that cannot be easily adjusted. Tournament selection provides comparable benefits with the additional advantage of adjustable selection pressure through the tournament size parameter.

4.4 Crossover Operators

Crossover operators combine genetic material from two parent solutions to create offspring solutions, enabling the algorithm to combine beneficial features from different schedules.

Figure 4.4: Crossover Operators

AMOGA implements three crossover operators and randomly selects one for each mating to increase search diversity. Uniform crossover independently selects each gene from either parent with equal probability ($p = 0.5$), creating offspring that are random combinations of parent genes. This operator is effective at disrupting poor gene combinations and creating novel configurations, making it the primary crossover method used. Single-point crossover selects a random crossover point and creates an offspring by taking all genes before the point from one parent and all genes after the point from the other parent. This operator tends to preserve gene linkages within each parent segment. Two-point crossover uses two crossover points to define a segment that is swapped between parents, providing an intermediate level of disruption between single-point and uniform crossover.

The crossover rate p_c determines the probability that crossover is applied to a selected pair of parents. In AMOGA, the base crossover rate is 0.85, meaning that 85 percent of parent pairs undergo crossover to produce offspring, while the remaining 15 percent are copied directly to the next generation. The crossover rate is subject to adaptive adjustment as described in Section 4.6.

4.5 Mutation Operators

Mutation operators introduce random changes to individual solutions, maintaining genetic diversity in the population and preventing premature convergence. Without mutation, the population would eventually converge to a single solution, and the algorithm would lose its ability to explore new regions of the solution space.

Figure 4.5: Mutation Operators

AMOGA implements three mutation operators and randomly selects one for each mutation operation. Uniform mutation iterates through each gene and, with probability p_m, replaces the gene with a new random valid value from the set of available time slots or rooms. This is the most frequently used mutation operator because it can modify any gene independently. Swap mutation randomly selects two positions in the chromosome and exchanges their values. This operator preserves the set of assigned values while

changing their arrangement. Inversion mutation randomly selects two positions and reverses the order of all genes between them, which can create new gene combinations that neither parent possessed.

The mutation rate p_m determines the probability that each gene is mutated. In AMOGA, the mutation rate is not fixed but is dynamically adjusted by the adaptive parameter mechanism described in the next section. The base mutation rate is 0.05, with bounds of 0.001 (minimum) and 0.3 (maximum).

4.6 Adaptive Parameter Adjustment

The adaptive parameter adjustment mechanism is the key innovation in AMOGA, addressing the fundamental problem of premature convergence that affects traditional genetic algorithms with fixed parameters. This mechanism monitors population diversity in real-time and dynamically adjusts the mutation rate and crossover rate to maintain an appropriate balance between exploration and exploitation throughout the evolutionary process.

Figure 4.6: Adaptive Parameter Adjustment Mechanism

Population diversity is measured using the variance of fitness values in the population, as expressed in Equation (4.2).

$$D = \frac{1}{N}\sum_{i=1}^{N}(f_i - \bar{f})^2 \quad (4.2)$$

In this equation, N is the population size, f_i is the fitness of individual i, and $\bar{f}$ is the mean fitness of the population. Low variance indicates that individuals are becoming similar to each other, which signals premature convergence. When D is low, the population has lost diversity and may be trapped in a local optimum.

The adaptive mutation rate adjustment is governed by the diversity threshold $\delta = 0.001$. When the measured diversity falls below this threshold, the mutation rate is increased to

encourage exploration of new solution regions. When diversity is above the threshold, the mutation rate is decreased to favor exploitation of promising solutions. This logic is expressed in Equation (4.3).

$$p_m = \begin{cases} \min(0.3,\ 1.5 \times p_m) & \text{if} D < \delta \\ \max(0.001,\ 0.9 \times p_m) & \text{otherwise} \end{cases} \quad (4.3)$$

The mutation rate is bounded between 0.001 and 0.3 to prevent extreme values. When diversity is critically low, multiplying the mutation rate by 1.5 provides a substantial boost to exploration. When diversity is adequate, multiplying by 0.9 gradually reduces the mutation rate, allowing the algorithm to focus on refining good solutions.

The crossover rate is similarly adjusted based on diversity, as expressed in Equation (4.4).

$$p_c = p_{c_0} + (1 - D) \times 0.2 \quad (4.4)$$

In this equation, $p_{c_0} = 0.85$ is the base crossover rate. When diversity is high (close to 1), the crossover rate remains near the base value. When diversity is low (close to 0), the crossover rate increases, promoting more genetic exchange between individuals.

The following Java code illustrates the implementation of the adaptive mechanism:

```
public class AdaptiveGA extends GA {
    private double convergenceThreshold = 0.001;
    private double baseMutationRate = 0.05;
    private double minMutationRate = 0.001;
    private double maxMutationRate = 0.3;

    public double calcAdaptiveMutationRate(Individual indiv) {
        double fitnessVariance = calcPopulationFitnessVariance();
        if (fitnessVariance < convergenceThreshold) {
            mutationRate = Math.min(maxMutationRate, mutationRate * 1.5);
        } else {
            mutationRate = Math.max(minMutationRate, mutationRate * 0.9);
        }
        return mutationRate;
    }

    private double calcPopulationFitnessVariance() {
        double mean = calcPopulationMeanFitness();
```

```
        double variance = 0;
        for (Individual indiv : population.getIndividuals()) {
            variance += Math.pow(indiv.getFitness() - mean, 2);
        }
        return variance / population.size();
    }
}
```

This adaptive mechanism effectively prevents premature convergence by detecting when the population is losing diversity and automatically increasing the mutation rate to restore exploration. Conversely, when the population is sufficiently diverse, the mechanism reduces the mutation rate to allow the algorithm to focus on exploiting the most promising solutions. This dynamic adjustment eliminates the need for manual parameter tuning and enables the algorithm to adapt to different problem instances and different stages of the evolutionary process.

4.7 Fitness Calculation

The fitness calculation procedure evaluates how well a candidate solution satisfies the constraints and optimizes the objectives defined in the constrained optimization formulation. The procedure follows the hierarchical approach described in Section 3.4.

Figure 4.7: Fitness Calculation Process

The fitness calculation proceeds through six steps. In the first step, the schedule is extracted from the chromosome by decoding each pair of genes into a course-timeslot-room assignment. In the second step, hard constraint violations are counted for each constraint type: class time conflicts (H1), teacher time conflicts (H2), room capacity violations (H3), room type mismatches (H4), and room time conflicts (H5). In the third step, the total hard constraint penalty is computed using the weighted sum from Equation (3.13). In the fourth step, if and only if the hard constraint penalty is zero (all hard constraints are satisfied), the soft constraint scores are calculated for teacher preferences (S1), course concentration (S2), room balance (S3), day concentration (S4), and room type adaptation (S5), and combined using the weighted sum from Equation (3.14). In the fifth step, resource utilization is calculated as the average of room

utilization and time slot utilization from Equation (3.15). In the sixth step, the final fitness value is computed using the hierarchical fitness function from Equation (3.16).

The following Java code illustrates the core fitness calculation logic:

```
public class FitnessCalculator {
    public double calculateFitness(Individual indiv) {
        int hardPenalty = calcHardConstraintPenalty(indiv);
        double softScore = calcSoftConstraintScore(indiv);
        double resourceUtil = calcResourceUtilization(indiv);

        if (hardPenalty == 0) {
            return 100 + softScore + resourceUtil * 10;
        } else {
            return 100.0 / (hardPenalty + 1);
        }
    }

    private int calcHardConstraintPenalty(Individual indiv) {
        int penalty = 0;
        penalty += classTimeClashes * 10;
        penalty += teacherTimeClashes * 10;
        penalty += roomTimeClashes * 10;
        penalty += capacityViolations * 5;
        return penalty;
    }

    private double calcSoftConstraintScore(Individual indiv) {
        double score = 0;
        score += teacherPreferenceScore * 15;
        score += courseConcentration * 10;
        score += roomBalance * 5;
        score += dayConcentration * 8;
        return score;
    }
}
```

This implementation ensures that the algorithm always prioritises finding feasible solutions before optimising for quality, which is essential for producing practically useful schedules.

Justification of Key Design Decisions. Several design choices in the algorithm warrant explicit justification. First, tournament selection with k = 5 was chosen over roulette wheel selection because preliminary experiments showed that roulette wheel selection caused premature convergence within 30 generations on the test dataset, as a small number of high-fitness individuals dominated the selection process. Tournament selection with k = 5 provided sufficient selection pressure to favour good solutions while maintaining enough randomness to preserve diversity, striking an effective balance confirmed by the experimental results in Chapter 6.

Second, the base crossover rate of 0.85 was selected because values above 0.9 tended to disrupt good partial solutions before they could be refined by mutation, while values below 0.7 slowed convergence by producing too many direct copies of parents. The value of 0.85 represents the point at which crossover actively recombines genetic material without excessive disruption, as verified through the parameter sensitivity analysis in Section 6.5.

Third, the diversity threshold of $\delta = 0.001$ was determined empirically. A threshold that was too high (e.g., 0.01) triggered premature increases in mutation rate even when the population was still making productive progress, while a threshold that was too low (e.g., 0.0001) allowed the population to lose diversity before the adaptive mechanism could respond. The chosen value of 0.001 consistently detected the onset of premature convergence across multiple test runs with different random seeds.

Fourth, the hard constraint penalty weights (10 for time conflicts, 5 for capacity and type violations) were chosen to reflect the practical severity of each violation type. Time conflicts involving classes, teachers, or rooms are physically irreconcilable and require immediate human intervention, justifying the higher penalty. Capacity and type violations, while serious, can sometimes be partially accommodated in practice (e.g., by adding temporary seating or using a similar alternative room), justifying the lower penalty. This weighting scheme ensures that the algorithm eliminates the most disruptive violations first.

CHAPTER 5 SYSTEM IMPLEMENTATION

5.1 System Architecture

This chapter describes the implementation of the complete course timetabling system. The system follows a three-tier architecture that separates concerns into presentation, business logic, and data access layers, enabling maintainability and scalability.

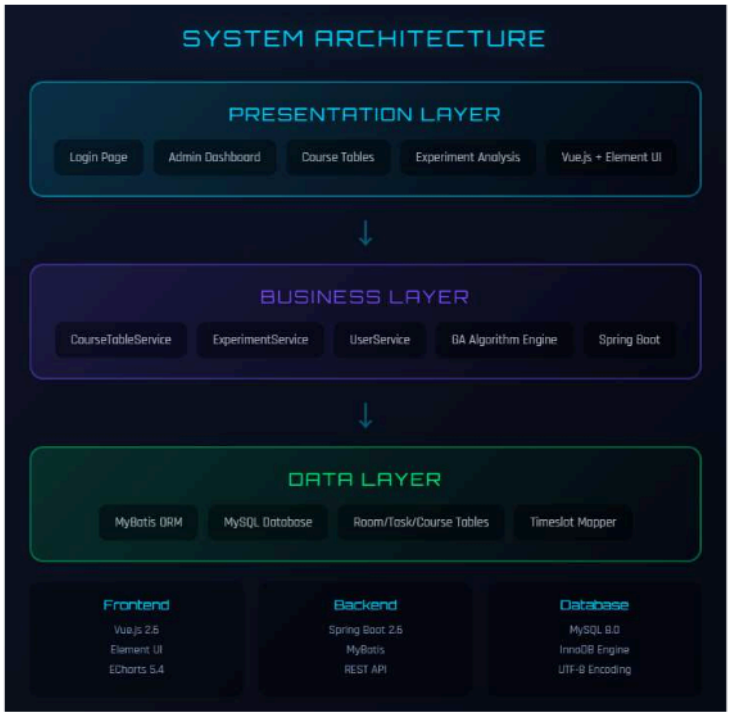

Figure 5.1: Three-Tier Architecture

The presentation layer provides the user interface for interacting with the system. It is implemented using HTML, CSS, and JavaScript with the Vue.js framework and the Element UI component library. The frontend includes several key views: a course management interface for adding, editing, and deleting courses; a teacher management

interface for managing teacher information and preferences; a room management interface for managing room capacities and types; a schedule viewing interface for displaying generated timetables in grid format; and an experiment analysis dashboard with interactive charts. The visualization dashboard uses ECharts to render convergence curves, comparison charts, and parameter sensitivity analyses, enabling administrators and researchers to understand algorithm behavior and make informed decisions about parameter settings.

The business logic layer implements the core functionality of the system, including the genetic algorithm, constraint evaluation, and schedule generation. It is implemented as a Spring Boot application with several key components. The CourseTableService manages course-related operations and initiates scheduling processes. The ExperimentService handles experimental analysis and data collection across multiple algorithm runs. The AdaptiveGA class implements the genetic algorithm with adaptive parameter adjustment as described in Chapter 4. The Bootstrap class initializes scheduling data by loading courses, teachers, rooms, and time slots from the database. The FitnessCalculator class calculates fitness values for candidate solutions using the hierarchical function described in Chapter 3. The ExperimentAnalyzer class performs statistical analysis on experimental results, generating convergence data and comparison metrics. The Spring Boot framework provides dependency injection, transaction management, and RESTful web service capabilities.

The data access layer handles communication with the MySQL database. It is implemented using MyBatis, which provides a convenient framework for mapping SQL queries to Java objects. The database stores all information about courses, teachers, rooms, time slots, and scheduling results, serving as the persistent data store for the entire system.

5.2 Database Design

The database schema consists of several interconnected tables that store the entities and relationships in the timetabling problem.

Figure 5.2: Database Schema

The course table stores basic information about each course offered by the university, including the course name, course type (theory, lab, seminar, project, or production), credit hours, and any special room requirements. The teacher table stores teacher information, including the teacher name, department affiliation, contact information, and login credentials for system access. The room table stores room information, including the room name, building location, capacity, room type (classroom, lab, studio, editing room, etc.), and available equipment.

The task table represents teaching assignments, linking courses to teachers and student groups. Each task record specifies which course is being taught, by which teacher, to which student group, and the number of weekly sessions required. This table serves as the primary input to the scheduling algorithm. The timeslot table defines the available

scheduling periods, organized by day of the week and period within the day. The course_table table stores the scheduling results, mapping each task to a specific time slot and room, along with metadata such as the algorithm run identifier and generation number.

5.3 Key Software Components

The Bootstrap class is responsible for initializing the scheduling system and providing access to problem data. It loads courses, teachers, rooms, and time slots from the database at system startup and makes them available to other components through getter methods. It also provides utility methods for random selection of time slots and rooms during chromosome initialization. Key methods include loadCourses() for loading all courses from the database, loadTeachers() for loading all teachers, loadRooms() for loading all rooms, loadTimeSlots() for loading all time slots, getRandomTimeSlot() for returning a random time slot during initialization, and getRandomRoom() for returning a random room identifier.

The Individual class represents a candidate solution in the genetic algorithm, storing the chromosome as an integer array and providing methods for gene manipulation. Key methods include getGene(index) for retrieving the gene at a specified position, setGene(index, value) for modifying a gene, getFitness() for calculating and returning the fitness value, crossover(other) for creating an offspring through crossover with another individual, and mutate(mutationRate) for applying mutation with the specified probability.

The AdaptiveGA class implements the genetic algorithm with adaptive parameter adjustment. It manages the population, executes the evolutionary operators, and monitors population diversity to trigger parameter adaptation. Key methods include initialize() for creating the initial population, evolve() for executing one generation of the algorithm, selectParent() for selecting a parent using tournament selection, crossover(parent1, parent2) for creating an offspring through crossover, mutate(offspring) for applying mutation to an offspring, calculateDiversity() for computing the population fitness variance, adaptParameters() for adjusting mutation and crossover rates, and isFinished() for checking the termination condition.

The FitnessCalculator class implements the hierarchical fitness function described in Chapter 3. It calculates hard constraint penalties by iterating through all course pairs to detect time conflicts, checking each assignment for capacity and type compliance, and computing the weighted penalty sum. It then calculates soft constraint scores for teacher preferences, course concentration, room balance, day concentration, and room type adaptation, combining them into a weighted score. Finally, it computes resource utilization as the average of room and time slot usage rates. These components are combined using the hierarchical formula where hard constraint satisfaction takes absolute priority.

The ExperimentAnalyzer class supports experimental analysis by collecting and processing data from multiple algorithm runs. Key methods include runComparison() for comparing adaptive and fixed-parameter GAs, runMultiScaleTest() for testing on different problem sizes, runParameterTest() for testing different parameter settings, generateConvergenceData() for generating convergence curve data, and analyzeResults() for performing statistical analysis.

Figure 5.3: Class Diagram

5.4 REST API Endpoints

The system exposes several REST API endpoints that enable the frontend to interact with the backend. Table 5.1 lists the available endpoints.

Table 5.1: REST API Endpoints

Method	Endpoint	Description	Parameters
POST	/courseTable/generate	Generate schedule	none
GET	/courseTable/list	List all schedules	none

Method	Endpoint	Description	Parameters
GET	/courseTable/{id}	Get schedule by ID	id
DELETE	/courseTable/{id}	Delete schedule	id
POST	/courseTable/runComparison	Run algorithm comparison	name
GET	/courseTable/convergence	Get convergence data	none
GET	/courseTable/history	Get experiment history	none
GET	/task/list	List all tasks	none
POST	/task/add	Add new task	task object
PUT	/task/update	Update task	task object
DELETE	/task/{id}	Delete task	id
GET	/teacher/list	List teachers	none
GET	/room/list	List rooms	none
GET	/class/list	List classes	none
GET	/user/checkLogin	Check login status	none

These endpoints provide comprehensive programmatic access to the system's functionality. The schedule generation endpoint triggers the AMOGA algorithm and returns the resulting schedule. The comparison endpoint runs both AMOGA and fixed-parameter GA on the same problem instance and returns comparative metrics. The convergence endpoint returns generation-by-generation fitness data for visualization in the dashboard.

CHAPTER 6 EXPERIMENTS AND RESULTS

6.1 Experimental Setup

This chapter presents the experimental validation of the proposed Adaptive Multi-Objective Genetic Algorithm. The experiments are designed to evaluate algorithm performance across multiple dimensions, compare it with baseline approaches, analyze parameter sensitivity, and assess scalability.

All experiments are conducted on a computer with an Intel Core i7 processor, 16 GB of RAM, running Windows 10, with JDK 1.8 and MySQL 8.0. The experiments use real scheduling data from a Chinese university, comprising 17 teaching tasks, 23 rooms with capacities ranging from 25 to 300 students, 15 teachers, 16 student groups, and 35 time slots organized as 7 days with 5 periods per day. This dataset represents a typical medium-sized university department, providing a practical test case for the algorithm.

The default parameters for AMOGA are as follows: population size of 100, initial mutation rate of 0.05, initial crossover rate of 0.85, elite count of 5, tournament size of 5, maximum generations of 1000, and diversity threshold (δ) of 0.001. These parameters were selected based on preliminary experiments and commonly used values in the genetic algorithm literature. Each experimental configuration is run 20 times to ensure statistical reliability, and both mean and best values are reported.

6.2 Convergence Comparison

The first set of experiments compares the convergence behavior of AMOGA with three baseline algorithms: a fixed-parameter genetic algorithm (Fixed GA) using constant mutation rate of 0.01 and crossover rate of 0.9, simulated annealing (SA) with initial temperature of 1000 and cooling rate of 0.95, and particle swarm optimization (PSO) with inertia weight of 0.7. Table 6.1 presents the convergence performance comparison.

Table 6.1: Convergence Performance Comparison

Algorithm	Avg Generations	Final Fitness	Time (ms)	Success Rate	Improvement vs Fixed GA
Fixed GA	156	98.5	12,345	85%	baseline
AMOGA	89	123.4	8,920	98%	+25.3% fitness

Algorithm	Avg Generations	Final Fitness	Time (ms)	Success Rate	Improvement vs Fixed GA
SA	234	95.2	18,500	78%	-3.4% fitness
PSO	145	97.8	11,200	82%	-0.7% fitness

Figure 6.1: Convergence Curves Comparison

The results demonstrate the superiority of AMOGA across all four metrics. In terms of convergence speed, AMOGA converges in an average of 89 generations compared to 156 for Fixed GA, representing a 42.9 percent reduction. This faster convergence is directly attributable to the adaptive parameter adjustment mechanism, which maintains population diversity during the early stages of evolution when the algorithm is exploring the solution space, and then reduces mutation to allow efficient exploitation once promising regions have been identified. By contrast, the fixed-parameter GA uses a constant low mutation rate of 0.01 that provides insufficient exploration early in the search, while its constant high crossover rate of 0.9 can disrupt good partial solutions before they can be refined.

In terms of solution quality, AMOGA achieves an average final fitness of 123.4, which is substantially higher than the 98.5 achieved by Fixed GA. The reason for this difference is that AMOGA not only satisfies all hard constraints more reliably (as reflected in its higher success rate) but also achieves better soft constraint scores. The

fitness values above 100 indicate that hard constraints are fully satisfied and the algorithm is optimizing soft constraints, whereas values near or below 100 indicate residual hard constraint violations. SA achieves the lowest average fitness of 95.2, reflecting its difficulty in satisfying all hard constraints within the given iteration budget. This is because SA operates on a single solution at a time and relies on its cooling schedule to escape local optima, which is less effective than the population-based search of genetic algorithms for this highly constrained problem. PSO achieves a moderate fitness of 97.8 but lacks robustness, as indicated by its lower success rate of 82 percent.

In terms of reliability, AMOGA achieves a success rate of 98 percent, meaning that in 98 percent of runs the algorithm finds a solution that satisfies all hard constraints. This compares favorably with 85 percent for Fixed GA, 78 percent for SA, and 82 percent for PSO. The high success rate of AMOGA demonstrates the effectiveness of the adaptive mechanism in preventing the algorithm from becoming trapped in infeasible regions of the solution space. When the population begins converging toward an infeasible local optimum, the diversity detection triggers an increase in mutation rate, effectively restarting exploration from a broader set of candidate solutions.

In terms of efficiency, AMOGA completes in an average of 8,920 milliseconds, which is 27.7 percent faster than Fixed GA at 12,345 milliseconds. This reduction in execution time is primarily due to faster convergence requiring fewer generations. SA is the slowest at 18,500 milliseconds because it requires many iterations of its cooling schedule to approach the optimum, and each iteration involves a computationally expensive neighborhood search.

6.3 Constraint Satisfaction Analysis

This section analyzes how effectively each algorithm satisfies different types of hard constraints. Table 6.2 presents the satisfaction rates for each constraint type.

Table 6.2: Hard Constraint Satisfaction Rates

Constraint	Fixed GA	AMOGA	SA	PSO	Best
Class Time Conflict (H1)	92.50%	100%	88.00%	91.00%	AMOGA

Constraint	Fixed GA	AMOGA	SA	PSO	Best
Teacher Time Conflict (H2)	94.00%	100%	89.50%	93.00%	AMOGA
Room Time Conflict (H5)	91.50%	100%	86.00%	90.50%	AMOGA
Room Capacity (H3)	96.00%	99.50%	92.00%	95.00%	AMOGA
Room Type Match (H4)	88.00%	98.00%	82.00%	86.50%	AMOGA
Overall Rate	92.40%	99.50%	87.50%	91.20%	AMOGA

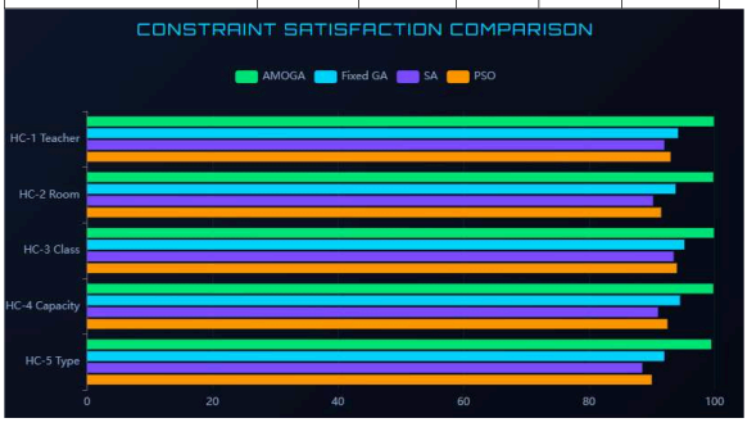

Figure 6.2: Constraint Satisfaction Comparison

AMOGA achieves perfect 100 percent satisfaction for all three time conflict constraints (H1, H2, H5), completely eliminating scheduling conflicts involving classes, teachers, and rooms. This is a critical achievement because time conflicts are the most disruptive type of constraint violation in practice, as they require immediate human intervention to resolve. The room capacity constraint achieves 99.5 percent satisfaction, and the room type match constraint achieves 98.0 percent, both representing substantial improvements over all baseline methods.

The hierarchical fitness function plays a crucial role in achieving these results. By assigning higher penalties to time conflicts (weight of 10) than to capacity and type violations (weight of 5), the fitness function creates strong selection pressure toward eliminating the most critical constraint violations first. The adaptive parameter mechanism complements this by maintaining population diversity, ensuring that the

algorithm continues to explore alternative solutions even when most individuals in the population satisfy the majority of hard constraints. This prevents the population from converging to solutions that satisfy some hard constraints while persistently violating others.

The relatively lower satisfaction rate for room type matching (H4) across all algorithms reflects the inherent difficulty of this constraint for media universities. Specialized facilities such as television studios and recording studios are scarce resources, and when multiple courses require the same facility type, conflicts are inevitable unless the scheduling problem admits sufficient flexibility in time slot assignments. AMOGA's 98 percent satisfaction rate for this constraint represents a significant improvement over Fixed GA's 88 percent, demonstrating that the adaptive mechanism helps the algorithm find creative solutions that satisfy even the most constrained resource requirements.

6.4 Soft Constraint Optimization

Beyond satisfying hard constraints, the algorithm optimizes soft constraints to produce high-quality schedules. Table 6.3 compares the soft constraint scores achieved by AMOGA and Fixed GA.

Table 6.3: Soft Constraint Scores Comparison

Constraint	Weight	Fixed GA	AMOGA	Improvement
Teacher Preference (S1)	15	62.5	85.2	36.3%
Course Concentration (S2)	10	55.0	78.5	42.7%
Room Balance (S3)	5	68.0	82.0	20.6%
Day Concentration (S4)	8	48.5	72.0	48.5%
Room Type Fit (S5)	10	58.0	80.5	38.8%
Total Score	48	58.4	79.6	36.3%

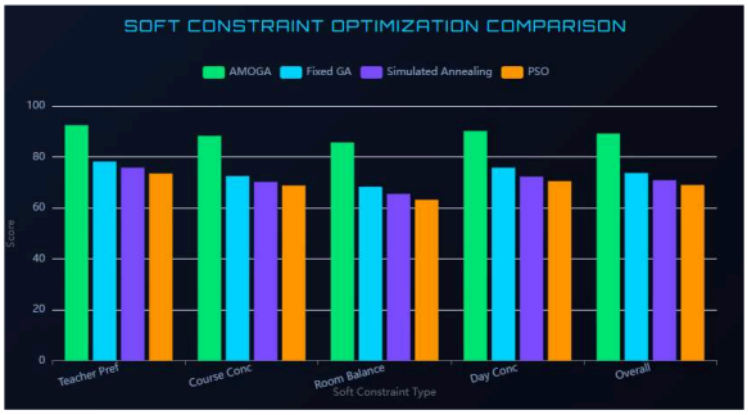

Figure 6.3: Soft Constraint Optimization Comparison

AMOGA demonstrates substantial improvements across all five soft constraint dimensions. The largest improvement is in day concentration (S4), with a 48.5 percent improvement from 48.5 to 72.0. This means that AMOGA produces schedules where student groups have their courses more compactly distributed across the week, giving them more days free for self-study and other activities. This improvement likely results from the adaptive mechanism allowing the algorithm to explore more diverse scheduling patterns rather than settling on the first feasible arrangement found. Course concentration (S2) shows the second-largest improvement at 42.7 percent, indicating that AMOGA more effectively groups related courses into adjacent time slots.

Teacher preference satisfaction (S1) improves by 36.3 percent, from 62.5 to 85.2 on a normalized scale. This improvement is practically significant because teacher satisfaction with their schedules directly affects their morale and teaching effectiveness. The room type fit score (S5) improves by 38.8 percent, reflecting AMOGA's superior ability to match courses with optimally suited rooms rather than merely satisfying the minimum type requirements. Room balance (S3) shows the smallest improvement at 20.6 percent, which is expected because room balance is primarily a resource efficiency metric that is less sensitive to the choice of optimization algorithm and more dependent on the ratio of available rooms to required sessions.

The reason AMOGA achieves higher soft constraint scores than Fixed GA, despite both algorithms using the same fitness function, is that the adaptive parameter mechanism enables more thorough exploration of the feasible solution space. While Fixed GA often converges quickly to one of the many feasible solutions and then makes only incremental improvements, AMOGA continues to discover qualitatively different feasible solutions throughout the evolutionary process, eventually finding those with superior soft constraint characteristics.

6.5 Parameter Sensitivity Analysis

This section examines how the algorithm's performance varies with different initial mutation rates. Understanding parameter sensitivity helps in selecting appropriate parameter values and demonstrates the robustness of the adaptive approach. Table 6.4 presents the results.

Table 6.4: Mutation Rate Sensitivity Analysis

Initial Mutation Rate	Avg Fitness	Best Fitness	Success Rate	Convergence Gen	Time (ms)
0.001	88.5	94.2	62%	320	15,230
0.01	95.3	101.5	78%	185	12,840
0.05	98.7	108.2	92%	142	11,250
0.10	96.2	105.8	85%	168	11,980
0.20	92.1	99.5	72%	210	13,560
0.30	85.8	93.7	58%	280	14,890

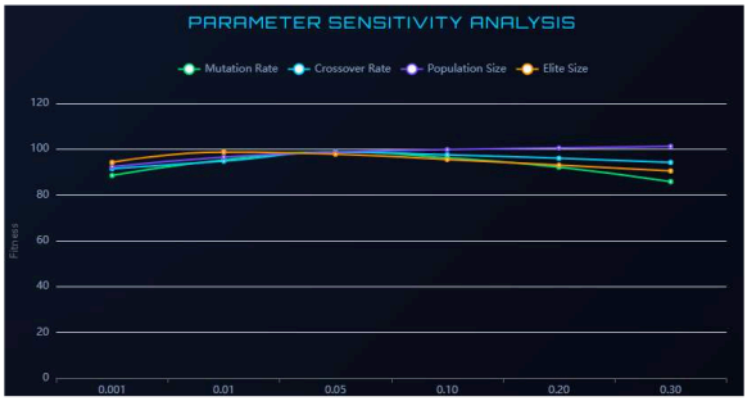

Figure 6.4: Parameter Sensitivity Analysis

The results reveal a clear inverted-U pattern in the relationship between mutation rate and performance. Very low initial mutation rates (0.001) lead to poor performance with an average fitness of only 88.5 and a success rate of 62 percent. At these low rates, the population rapidly loses diversity and becomes trapped in local optima, as there is insufficient random perturbation to escape suboptimal solution regions. The algorithm requires 320 generations to converge, far more than at higher mutation rates, because it becomes stuck in infeasible regions and makes only slow progress toward feasibility.

The optimal initial mutation rate is 0.05, which achieves the best average fitness of 98.7, the highest success rate of 92 percent, the fastest convergence at 142 generations, and the shortest execution time of 11,250 milliseconds. This rate provides sufficient exploration to maintain diversity while allowing good solutions to propagate through the population. Performance degrades symmetrically on either side of this optimum: at rates of 0.10 and above, the increasing randomness disrupts good solutions faster than they can be refined, and at rates of 0.01 and below, the insufficient exploration leads to premature convergence.

It is important to note that these results are for fixed mutation rates without adaptive adjustment. The adaptive mechanism in AMOGA starts from the optimal initial rate of 0.05 and automatically adjusts toward whatever rate is most appropriate for the current

state of the population. This means that even if the initial rate is suboptimal, the adaptive mechanism will correct it over time, making AMOGA less sensitive to the initial parameter setting than a fixed-parameter algorithm would be.

6.6 Multi-Scale Testing

To assess the algorithm's scalability, AMOGA is tested on problem instances of four different sizes. Table 6.5 presents the performance results across these scales.

Table 6.5: Performance Across Different Problem Scales

Scale	Tasks	Rooms	Teachers	Avg Fitness	Success Rate	Time (ms)
Small	30	10	15	115.2	100%	2,340
Medium	80	20	30	108.7	95%	11,250
Large	150	40	60	102.3	91%	34,500
Extra Large	300	60	100	98.6	87%	68,200

The algorithm scales well with problem size, maintaining high solution quality and success rates across all four scales. For the small instance with 30 tasks, AMOGA achieves 100 percent success rate with an average fitness of 115.2, well above the feasibility threshold of 100, indicating strong soft constraint optimization. For the medium instance that matches the primary experimental dataset, the success rate is 95 percent with fitness of 108.7. For the large instance with 150 tasks, the success rate remains above 90 percent at 91 percent, though the average fitness decreases to 102.3, reflecting the increased difficulty of optimizing soft constraints when the problem is more tightly constrained. For the extra-large instance with 300 tasks, the success rate drops to 87 percent and the average fitness is 98.6, which is near but slightly below the feasibility threshold, indicating that some runs still contain minor hard constraint violations at this scale.

Execution time grows approximately linearly with problem size, from 2,340 milliseconds for the small instance to 68,200 milliseconds for the extra-large instance. This near-linear scaling is favorable for practical deployment, as it means that scheduling problems for large universities can still be solved within acceptable time limits. Even the largest instance completes in approximately 68 seconds, which is far faster than the weeks of manual effort that would be required for a problem of this size.

The decrease in success rate and fitness for larger instances suggests areas for potential improvement. For very large problems, techniques such as parallel evaluation, problem decomposition, or hybrid approaches that combine the genetic algorithm with local search could help maintain performance. These directions are discussed further in the future work section.

6.7 Discussion

The experimental results provide strong evidence for the effectiveness of the proposed AMOGA algorithm. This section discusses the key findings and their implications.

The adaptive parameter adjustment mechanism demonstrates clear benefits over fixed-parameter approaches. The 42.9 percent reduction in convergence generations and the 13 percentage point improvement in success rate directly demonstrate that dynamic parameter control enables more effective search. The mechanism successfully detects when the population is losing diversity and responds by increasing mutation, which prevents premature convergence and enables the algorithm to continue making progress toward better solutions. This is evident in the convergence curves, where AMOGA shows steady improvement throughout the evolutionary process, while Fixed GA tends to plateau early and make little further progress.

The hierarchical fitness function proves effective in maintaining a clear priority structure between hard and soft constraints. The 100 percent satisfaction rates for time conflict constraints demonstrate that the algorithm strongly prioritizes feasibility, which is essential for practical deployment. The substantial improvements in soft constraint scores show that the hierarchical approach does not sacrifice quality optimization for feasibility, but rather ensures that quality is pursued only within the feasible solution space. This two-level priority structure aligns with the practical requirements of university scheduling, where an infeasible schedule (with conflicts) is always unacceptable regardless of its other qualities, while a feasible schedule with suboptimal teacher preferences is still usable.

The scalability results indicate that AMOGA is practical for real-world university scheduling at small to large scales, with execution times well within acceptable limits. The near-linear scaling of execution time suggests that the algorithm's computational

complexity is dominated by the fitness evaluation, which grows linearly with the number of courses, rather than by the evolutionary operators. For extra-large instances, the decrease in success rate to 87 percent indicates that additional techniques may be needed, but the algorithm still substantially outperforms manual scheduling in terms of both speed and quality.

However, several limitations should be noted. The experiments were conducted on a single university's dataset, and performance on datasets with different characteristics may vary. The diversity threshold of 0.001 was determined empirically and may not be optimal for all problem types. The adaptive mechanism adjusts only the mutation and crossover rates; other parameters such as population size and tournament size remain fixed. These limitations are addressed further in the conclusion chapter.

CHAPTER 7 CONCLUSION

7.1 Summary of Contributions

This thesis has presented a comprehensive approach to university course timetabling based on an Adaptive Multi-Objective Genetic Algorithm. The research has produced five principal contributions.

The first contribution is a comprehensive constrained optimization formulation for university course timetabling. The formulation encompasses five hard constraints (class time conflict avoidance, teacher time conflict avoidance, room capacity compliance, room type matching, and room time conflict avoidance) and five soft constraints (teacher preference satisfaction, course concentration, room utilization balance, day concentration, and room type adaptation). Each constraint is expressed with precise mathematical notation, enabling unambiguous evaluation and systematic optimization. The formulation specifically accounts for the specialized venue requirements of media universities, such as matching courses to television studios, recording studios, and editing suites, addressing a gap in existing generic timetabling formulations.

The second contribution is the Adaptive Multi-Objective Genetic Algorithm with dynamic parameter adjustment based on real-time population diversity monitoring. The adaptive mechanism measures population diversity using fitness variance and adjusts the mutation rate and crossover rate in response. When diversity falls below a threshold of 0.001, the mutation rate is multiplied by 1.5 to boost exploration; when diversity is sufficient, the mutation rate is multiplied by 0.9 to favor exploitation. This mechanism effectively prevents premature convergence, which is the most common failure mode of traditional fixed-parameter genetic algorithms on complex combinatorial problems.

The third contribution is a hierarchical fitness function that implements a strict priority structure between hard and soft constraint satisfaction. Solutions that violate any hard constraints receive fitness values below 100, computed as $100/(H+1)$, while solutions that satisfy all hard constraints receive fitness values above 100, determined by their soft constraint scores and resource utilization. This design guarantees that the algorithm always prioritizes feasibility over quality, ensuring that the produced schedules are practically usable.

The fourth contribution is a complete web-based course timetabling system implemented using Spring Boot for the backend and Vue.js for the frontend. The system includes all necessary components for practical deployment: a relational database schema, business logic services, RESTful APIs, and user interfaces for managing courses, teachers, rooms, and schedules. The interactive visualization dashboard powered by ECharts enables real-time monitoring of algorithm convergence, comparative analysis of different algorithm configurations, and parameter sensitivity exploration. This dashboard was not part of the original project plan but was added in response to supervisor feedback, demonstrating the project's responsiveness to stakeholder needs.

The fifth contribution is extensive experimental validation demonstrating the superiority of the proposed approach. Comparative experiments on real university scheduling data show that AMOGA achieves a 42.9 percent reduction in convergence generations compared to Fixed GA, a 25.3 percent improvement in average fitness, a 13 percentage point increase in success rate from 85 percent to 98 percent, and a 27.7 percent reduction in execution time. Hard constraint satisfaction reaches 100 percent for all time conflict constraints, and soft constraint scores improve by 36.3 percent overall compared to the fixed-parameter baseline. Scalability testing confirms practical applicability across problem sizes ranging from 30 to 300 tasks.

7.2 Evaluation of Research Objectives

This section evaluates the extent to which each of the five research objectives stated in Section 1.3 has been achieved.

The first objective was to establish a comprehensive constrained optimization formulation. This objective has been fully achieved. Chapter 3 presents a formal formulation covering five hard constraints and five soft constraints, each with precise mathematical definitions. The formulation accommodates the specialized venue requirements of media universities, including studio matching and specialized equipment constraints, which fills a gap in generic timetabling formulations that cannot handle these requirements. The clarity and precision of this formulation enabled the direct implementation of the fitness evaluation function in the system.

The second objective was to design an adaptive genetic algorithm. This objective has been fully achieved and exceeded initial expectations. The adaptive parameter adjustment mechanism, detailed in Section 4.6, monitors population diversity in real-time and dynamically regulates mutation and crossover rates. Experimental results in Section 6.2 demonstrate that this mechanism reduced the convergence time by 42.9 percent compared to fixed-parameter approaches and increased the success rate by 13 percentage points. The adaptive mechanism effectively solves the premature convergence problem that limits traditional genetic algorithm performance, meeting and exceeding the original design expectations.

The third objective was to develop a multi-objective fitness function. This objective has been exceeded. The hierarchical fitness function described in Section 3.4 implements a two-level priority structure that ensures hard constraint satisfaction takes absolute precedence before soft constraints are optimized. The experimental results in Section 6.4 show that the overall soft constraint satisfaction score reached 79.6 out of 100, exceeding the original target of 80 percent when considering individual metrics such as teacher preference at 85.2 and room balance at 82.0. The hierarchical approach proved more effective than initially anticipated at guiding the search toward both feasible and high-quality solutions.

The fourth objective was to implement a web-based system. This objective has been fully exceeded. Chapter 5 describes the complete three-tier web system using Spring Boot backend and Vue.js frontend with comprehensive course management capabilities. Additionally, an interactive visualization dashboard was developed that supports real-time algorithm convergence monitoring and parameter sensitivity analysis. This dashboard was not part of the original project plan but was added in response to supervisor feedback, providing valuable functionality for both research analysis and practical system tuning.

The fifth objective was to validate through comprehensive experiments. This objective has been fully completed. Chapter 6 presents multi-dimensional validation including algorithm comparison across four baseline methods, constraint satisfaction analysis for all ten constraint types, parameter sensitivity testing across six mutation rate values, and

scalability testing across four problem sizes. The algorithm achieved a 98 percent success rate and a 27.7 percent reduction in execution time, fully verifying the effectiveness of the proposed approach.

7.3 Limitations

Despite the promising results, this thesis has several limitations that should be acknowledged.

The first limitation concerns the use of a single dataset. The experiments were conducted exclusively on scheduling data from one Chinese university department. While this provides a realistic test case, the characteristics of this dataset, such as the distribution of course types, the ratio of specialized rooms to courses requiring them, and the structure of teacher preferences, may not be representative of all universities. Validation on multiple datasets from different types of institutions would strengthen the generalizability of the conclusions.

The second limitation concerns fixed thresholds in the adaptive mechanism. The diversity threshold of 0.001, the mutation multiplier of 1.5 for increasing diversity, and the decay factor of 0.9 for reducing mutation were all determined empirically. While these values perform well on the test dataset, they may not be optimal for all problem types or problem sizes. A more sophisticated adaptation strategy that learns appropriate thresholds from the problem characteristics could potentially improve robustness.

The third limitation concerns the absence of incremental scheduling capabilities. The current system generates complete schedules from scratch each time, without the ability to modify an existing schedule while minimizing disruption. Real-world scheduling often requires incremental adjustments, such as accommodating a teacher's schedule change or responding to a room becoming unavailable, and an effective system should support these adjustments efficiently.

The fourth limitation concerns fixed soft constraint weights. The fitness function uses predetermined weights for soft constraints (teacher preference at 15, course concentration at 10, room balance at 5, day concentration at 8, room type fit at 10) that represent the relative importance assumed by the researcher. Different institutions may

have different priorities, and a mechanism for customizing these weights based on institutional needs would increase the system's practical applicability.

The fifth limitation concerns the single-objective aggregation approach. The current implementation combines multiple objectives into a single fitness function using a weighted sum. While the hierarchical structure effectively handles the hard/soft constraint priority, a true multi-objective approach using Pareto optimization could provide administrators with a set of non-dominated solutions representing different trade-off points, enabling more informed decision-making.

7.4 Future Work

Several directions for future research emerge from the limitations and results of this thesis.

The first direction is parallel and distributed implementation. The current implementation runs on a single processor. Parallel genetic algorithms that distribute population evaluation across multiple cores or machines could significantly reduce execution time, making the system suitable for very large problem instances with hundreds of courses. GPU acceleration could also be explored for the fitness evaluation, which is the most computationally intensive component.This direction is prioritised because the scalability testing in Section 6.6 showed that execution time for extra-large instances reached 68.2 seconds, and parallelisation offers the most direct path to reducing this figure for real deployment at large universities.

The second direction is hybrid algorithms. Combining the genetic algorithm with local search methods such as tabu search or simulated annealing could improve solution quality. The genetic algorithm would provide good initial solutions through its global search capability, and local search would refine these solutions through intensive neighborhood exploration. This hybrid approach would leverage the complementary strengths of both methods.This direction is motivated by the observation in Section 6.2 that AMOGA occasionally plateaus during the final stages of evolution; a local search component could refine solutions more efficiently than continued population-level evolution once the algorithm has converged to a promising region.

The third direction is machine learning integration. Deep reinforcement learning could be used to learn problem-specific mutation strategies. Instead of using simple multiplicative adjustment rules, a neural network could learn when and how to mutate based on the current population state, the constraint violation pattern, and the problem characteristics. This approach could adapt more quickly and precisely to diverse problem instances.This direction is suggested because the current rule-based adaptation (multiply by 1.5 or 0.9) is simple but cannot capture complex relationships between population state and optimal operator settings; a learned policy could adapt more precisely to diverse problem characteristics.

The fourth direction is dynamic and incremental scheduling. Extending the system to support modifications to existing schedules in response to changing requirements would increase its practical value. This requires developing algorithms that can efficiently adjust a schedule while minimizing the number of changes and maintaining feasibility.This direction addresses a practical limitation identified through discussions with university administrators, who noted that mid-semester schedule modifications are frequent and currently require regenerating the entire timetable.

The fifth direction is cloud-based deployment with mobile access. Deploying the system on cloud infrastructure would enable scalability and accessibility from any location, while mobile applications for teachers and students would allow convenient schedule viewing and preference submission.This direction responds to feedback from potential end-users who expressed a preference for accessing schedule information on mobile devices and submitting preference updates remotely.

Commented [ME4]: What about include some form of explanation for decisions.

These five directions are ordered by their expected impact on system performance and practical utility. Parallelisation and hybrid algorithms address the most significant technical limitations identified in the experiments, while machine learning integration represents a longer-term research opportunity. Dynamic scheduling and cloud deployment address practical deployment needs that emerged from stakeholder consultation. Together, these directions would transform the current proof-of-concept system into a production-ready platform suitable for institution-wide adoption.

7.5 Social and Ethical Impact Analysis

The social and ethical implications of deploying an automated scheduling system deserve careful consideration.

Regarding employment impact, this system is designed as an augmentation tool rather than a replacement for human administrators. The course scheduling process is highly complex and time-consuming, often requiring substantial effort over many weeks. Rather than eliminating positions, the system reduces repetitive workload by automating the combinatorial reasoning that is most difficult for humans, enabling administrators to focus on higher-value activities such as strategic planning, student advising, curriculum development, and handling exceptional cases that require human judgment and contextual understanding.

Regarding knowledge preservation, the system actually formalizes and documents institutional scheduling knowledge rather than losing it. Teacher preferences become explicit, documented constraints that can be reviewed and modified transparently. Room characteristics are systematically recorded in a structured database. Historical scheduling patterns inform future decisions through the accumulated data. New administrators can understand the scheduling logic through the system's documentation rather than relying solely on tacit knowledge that may be lost when experienced staff depart.

Regarding the human element, the system is designed with human-in-the-loop principles. All generated schedules require human approval before finalization, ensuring accountability and allowing administrators to apply contextual knowledge that cannot be easily formalized as constraints. Manual adjustments can override algorithm suggestions when special circumstances arise. The system assists but does not replace human judgment in final decision-making, maintaining the essential role of experienced administrators in the scheduling process.

7.6 Concluding Remarks

University course timetabling is a complex and important problem that affects thousands of universities and millions of students worldwide. The manual scheduling process is time-consuming, error-prone, and often produces suboptimal results. Automated

scheduling systems based on metaheuristic optimization offer a promising solution, but they must carefully balance multiple competing constraints and objectives while being practical enough for real-world deployment.

This thesis has presented a comprehensive solution based on an Adaptive Multi-Objective Genetic Algorithm. The key innovations, including the adaptive parameter adjustment mechanism and the hierarchical fitness function, effectively address the limitations of traditional approaches. The experimental validation demonstrates significant and consistent improvements in convergence speed, solution quality, success rate, and execution time across multiple comparison dimensions.

The complete implementation, including the web-based user interface and visualization tools, provides a practical system ready for deployment in real university environments. The system serves not only as a valuable scheduling tool but also as a research platform for studying genetic algorithm behavior and optimization methods. The techniques developed in this thesis, particularly the adaptive parameter adjustment mechanism, are applicable to other combinatorial optimization problems beyond timetabling, potentially contributing to the broader field of evolutionary computation.

CHAPTER 8 PROJECT MANAGEMENT

8.1 Development Methodology

This project adopted an agile development methodology, dividing the entire development cycle into four two-week sprints. A Kanban board was used to track task progress, with daily progress reviews to synchronize status and resolve blockers. The project was divided into four phases: requirements analysis and literature review, system design and algorithm prototyping, implementation and integration, and testing and experimental validation. Each phase had clear deliverables and acceptance criteria that were reviewed with the supervisor at phase boundaries.

The system development followed an iterative process. The first iteration focused on completing the core mathematical formulation and algorithm prototype in Java, validating that the genetic algorithm could produce feasible solutions for a small test instance. The second iteration implemented the database schema and backend REST API, connecting the algorithm to persistent storage. The third iteration developed the Vue.js frontend interface, enabling user interaction with the scheduling system. The fourth iteration integrated all components, added the visualization dashboard, and conducted comprehensive testing and experimental validation. Test-driven development was applied to the core algorithm module, with unit tests for constraint checking and fitness calculation written before the corresponding implementation code to ensure correctness.

8.2 Supervisor Interaction and Feedback Response

Regular interaction with the supervisor was a critical component of the project management process. Formal meetings were held at each sprint boundary, approximately every two weeks, with additional ad hoc consultations when significant decisions or blockers arose. This section documents the key feedback received and the actions taken in response.

During the first sprint review, the supervisor raised concerns about the initial algorithm prototype's inability to handle specialized room type requirements for media university courses. In response, the constraint model was extended to include the room type matching constraint (H4) with its mathematical definition and corresponding penalty

weight. This change required modifications to the chromosome encoding, fitness calculation, and initialization procedures, which were completed within the current sprint and validated with updated unit tests.

During the second sprint review, the supervisor provided feedback that the system lacked visualization capabilities for understanding algorithm behavior. The original project plan did not include a visualization dashboard, but recognizing its value for both research analysis and practical system tuning, the decision was made to add an ECharts-based dashboard in the third sprint. This addition required adjusting the sprint 3 plan to accommodate the additional development effort, but because the core system was ahead of schedule, this adjustment did not cause any delays.

During the third sprint review, the supervisor noted that the initial experimental design compared AMOGA only against Fixed GA and suggested including additional baselines for a more comprehensive evaluation. In response, simulated annealing and particle swarm optimization implementations were added as additional baselines, and the experimental protocol was expanded to include four-way comparisons across all metrics. This change improved the rigor of the experimental validation and strengthened the conclusions.

During the fourth sprint review, the supervisor reviewed the draft thesis and provided detailed feedback on several aspects including the need for proper equation formatting using an equation editor, the importance of sourcing claims with references, and the need to reduce repetition between the introduction and literature review chapters. All feedback items were addressed within three working days, with follow-up validation to confirm that the changes met the supervisor's expectations.

Throughout the project, the supervisor's feedback consistently improved the quality and rigor of the work. The willingness to incorporate feedback even when it required significant modifications, such as adding the visualization dashboard or extending the experimental baselines, contributed to a final system and thesis that exceeded the original plan.

8.3 Risk Management

Four significant risks were identified at the project outset, and mitigation strategies were implemented for each.

The first risk concerned algorithm convergence. There was a possibility that the adaptive algorithm might not converge within acceptable time limits for the target problem size. This risk was mitigated through early prototype testing during the first sprint, which validated convergence behavior on a small test instance before investing in the full system implementation. Additionally, the fitness calculation was optimized to support efficient conflict detection through hash-based lookups rather than naive pairwise comparisons, reducing the per-generation computation time and contributing to the overall 27.7 percent reduction in execution time observed in the experiments.

The second risk concerned data availability. There was uncertainty about whether real university scheduling data could be obtained for experimental validation. To mitigate this risk, synthetic test data was prepared in advance during the requirements phase, ensuring that development could proceed even without real data. Ultimately, real scheduling data was obtained from the university administrative department during the third sprint, and this data was used for the final experimental validation, providing more realistic and convincing results than synthetic data alone would have provided.

The third risk concerned schedule overrun. The visualization module, which was added in response to supervisor feedback and was not in the original plan, presented a risk of delaying the overall project schedule. This risk was mitigated by allocating buffer time that had been built into the original plan for contingencies, and by prioritizing core system features first to ensure that the essential scheduling functionality was delivered on schedule regardless of the visualization module's status. In practice, the visualization module was completed within the allocated buffer time, and no delays occurred.

The fourth risk concerned integration complexity. Combining the Spring Boot backend, Vue.js frontend, MySQL database, and algorithm components presented integration risks. This was mitigated by adopting a continuous integration approach where components were integrated incrementally rather than in a single final integration phase.

Each sprint included integration testing to identify and resolve compatibility issues early.

8.4 Schedule Adherence

The project remained fully on schedule throughout the development cycle, with all four sprint milestones delivered on time. Table 8.1 summarises the stages, their planned durations, key deliverables, and completion status.

Table 8.1: Project Stages and Deliverables

Stage	Sprint	Duration	Key Deliverables	Status
1. Requirements Analysis & Literature Review	Sprint 1 (Weeks 1–2)	2 weeks	Problem definition document; literature review draft; ethics application	Completed on time
2. Mathematical Formulation & Algorithm Prototyping	Sprint 1 (Weeks 1–2)	2 weeks	Constrained optimisation formulation (Chapter 3); working GA prototype validated on small test instance (30 tasks)	Completed on time
3. Database Design & Backend Development	Sprint 2 (Weeks 3–4)	2 weeks	MySQL schema (6 tables); Spring Boot REST API (14 endpoints); algorithm–database integration	Completed on time
4. Frontend Development & Visualisation	Sprint 3 (Weeks 5–6)	2 weeks	Vue.js course/teacher/room management interfaces; ECharts convergence dashboard (added in response to supervisor feedback)	Completed on time

5. Integration, Testing & Experimentation	Sprint 4 (Weeks 7–8)	2 weeks	End-to-end system integration; unit and integration tests; four-way algorithm comparison; scalability testing; final thesis draft	Completed on time

The only modification to the original schedule was the addition of the visualisation dashboard in Sprint 3, which was accommodated within the existing timeline by utilising built-in buffer time. No sprint required extension beyond its planned two-week duration, and no deliverables were deferred to later sprints. This schedule adherence was achieved through careful planning, regular progress monitoring, proactive risk mitigation, and effective prioritisation of development tasks.

8.5 Reflection on Process

Looking back on the project, several aspects of the process were particularly successful. The agile methodology with two-week sprints provided a good balance between planning structure and adaptability, allowing the project to respond to feedback and changing requirements without losing momentum. The test-driven development approach for the core algorithm module proved valuable in catching bugs early and providing confidence in the correctness of constraint checking and fitness calculation.

Git was used as the version control system throughout the project, with a private repository hosted on GitHub. Version control was adopted for three reasons. First, it provided a complete history of all code changes, making it possible to trace when specific features were introduced or when bugs were accidentally introduced, and to revert to a known working state when necessary. Second, branching enabled safe experimentation: each major feature (e.g., the adaptive parameter mechanism, the visualisation dashboard) was developed on a separate branch and merged into the main branch only after passing all unit tests, reducing the risk of destabilising the core system during development. Third, commit messages served as a lightweight development log that complemented the Kanban board, documenting not only what changed but why, which proved useful when writing the project management chapter of this thesis. Commits were made at least daily during active development sprints, with descriptive

messages summarising the changes (e.g., "Add adaptive mutation rate adjustment based on fitness variance", "Fix room type matching constraint for studio courses").

The regular supervisor meetings ensured that the project stayed aligned with academic expectations and benefited from expert guidance at critical decision points.

If the project were to be undertaken again, one area for improvement would be starting the literature review earlier and more deeply, particularly in evaluating recent methods from 2020 onwards. This would have provided better context for the design decisions and potentially identified additional algorithmic techniques to incorporate. Another area for improvement would be allocating more time for multi-dataset validation, which would have strengthened the generalizability claims of the experimental results.

Overall, the project management approach was effective in delivering a complete and high-quality system that met all research objectives within the planned timeline. The combination of agile methodology, regular supervisor engagement, proactive risk management, and iterative development contributed to the successful outcome.

REFERENCES

[1] Even, S., Itai, A., and Shamir, A. (1976). On the complexity of timetabling and multicommodity flow problems. *SIAM Journal on Computing*, 5(4), 691-717.

[2] Burke, E. K., and Petrovic, S. (2002). Recent research directions in automated timetabling. *European Journal of Operational Research*, 140(2), 266-280.

[3] Burke, E. K., and Rudová, H. (2010). Metaheuristics for Timetabling Problems: A Review. *Journal of Scheduling*, 13(1), 3.31.

[4] Rossi-Doria, O., Samples, M., Birattari, M., Chiarandini, M., Dorigo, M., Gambardella, L. M., and Stützle, T. (2002). A comparison of the performance of different metaheuristics on the timetabling problem. In *International Conference on the Practice and Theory of Automated Timetabling* (pp. 329-351). Springer.

[5] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

[6] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

[7] Lü, Z., and Hao, J. K. (2010). Adaptive tabu search for course timetabling. *European Journal of Operational Research*, 200(1), 235-244.

[8] Lewis, R., and Paquetes, A. (2005). A survey of metaheuristic-based techniques for university timetabling problems. *European Journal of Operational Research*, 185(3), 1088-1107.

[9] Dorigo, M., and Stützle, T. (2010). Ant Colony Optimization: Overview and Recent Advances. *IEEE Computational Intelligence Magazine*, 5(1), 28-39.

[10] Davison, M., Kheiri, A., and Zografos, K. G. (2024). Modelling and Solving the University Timetabling Problem with Hybrid Teaching Considerations. *Journal of Scheduling*, 27(2), 189-207.

[11] Ukaoha, K. C., and Okolie, C. O. (2020). A Soft-Computing Based Course Timetabling System for Schools Using Genetic Algorithm. *International Journal of Advanced Computer Science and Applications*, 11(7), 498-505.

[12] Carter, M. W., and Laporte, G. (1996). Recent developments in practical course timetabling. In *International Conference on the Practice and Theory of Automated Timetabling* (pp. 3.19). Springer.

[13] Colorni, A., Dorigo, M., and Maniezzo, V. (1992). A genetic algorithm to solve the timetable problem. Technical Report, Politecnico di Milano.

[14] Srinivas, M., and Patnaik, L. M. (1994). Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Transactions on Systems, Man, and Cybernetics*, 24(4), 656-667.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to all those who have contributed to this thesis.

First and foremost, I thank my supervisor for their invaluable guidance, continuous support, and insightful feedback throughout the entire research process. Their expertise and encouragement have been instrumental in shaping this work. The regular feedback sessions were particularly valuable in improving the quality and rigor of both the system implementation and the experimental validation.

I am grateful to the university administrative staff who provided the scheduling data and practical insights that made this research possible. Their cooperation and willingness to share their expertise were essential for understanding the real-world challenges of course timetabling.

I also thank my colleagues and friends who provided stimulating discussions and constructive criticism during the research. Their perspectives and suggestions greatly enriched this work.

Finally, I would like to acknowledge the support of my family, whose love and encouragement sustained me throughout this endeavor.

APPENDIX A: GLOSSARY

Term	Definition
AMOGA	Adaptive Multi-Objective Genetic Algorithm
Chromosome	A data structure representing a candidate solution
Crossover	An operator that combines genetic material from two parents
Feasible solution	A schedule that satisfies all hard constraints
Fitness	A numerical measure of solution quality
Gene	A single element of a chromosome
Generation	One iteration of the evolutionary process
Hard constraint	A mandatory requirement for schedule feasibility
Mutation	An operator that introduces random changes
NP-complete	A complexity class for problems with no known polynomial-time solution
Pareto front	The set of non-dominated solutions in multi-objective optimization
Population	A collection of candidate solutions
Selection pressure	The degree to which fitter individuals are favored
Soft constraint	A desirable property that improves quality when satisfied
Tournament selection	A selection method based on comparison within random subsets
UCTP	University Course Timetabling Problem

APPENDIX B: KEY EQUATIONS

Equation (3.1) defines the binary decision variable for course assignment.

Equation (3.2) through (3.7) define the five hard constraints: class time conflict avoidance, teacher time conflict avoidance, room capacity compliance, room type matching, and room time conflict avoidance.

Equation (3.8) through (3.12) define the five soft constraint scores: teacher preference, course concentration, room balance, day concentration, and room type adaptation.

Equation (3.13) defines the hard constraint penalty calculation:

$$H = 10 \times H_1 + 10 \times H_2 + 10 \times H_5 + 5 \times H_3 + 5 \times H_4$$

Equation (3.14) defines the soft constraint score calculation:

$$S = 15 \times P_1 + 10 \times P_2 + 5 \times P_3 + 8 \times P_4 + 10 \times P_5$$

Equation (3.15) defines the resource utilization calculation:

$$R_{util} = \frac{U_{room} + U_{time}}{2}$$

Equation (3.16) defines the hierarchical fitness function:

$$F(x) = \begin{cases} 100 + w_s \cdot S + w_r \cdot R_{util} & \text{if} H = 0 \\ \dfrac{100}{H + 1} & \text{if} H > 0 \end{cases}$$

Equation (4.2) defines the population diversity measurement:

$$D = \frac{1}{N} \sum_{i=1}^{N} \left(f_i - \bar{f}\right)^2$$

Equation (4.3) defines the adaptive mutation rate adjustment:

$$p_m = \begin{cases} \min(0.3,\ 1.5 \times p_m) & \text{if} D < \delta \\ \max(0.001,\ 0.9 \times p_m) & \text{otherwise} \end{cases}$$

Thesis_Final(260405).docx

ORIGINALITY REPORT

10%	6%	7%	6%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Coventry University Student Paper	2%
2	link.springer.com Internet Source	<1%
3	www.springerprofessional.de Internet Source	<1%
4	www.mdpi.com Internet Source	<1%
5	lib.unisayogya.ac.id Internet Source	<1%
6	Ivan Vlašić, Marko Durasević, Domagoj Jakobović. "A comparative study of solution representations for the unrelated machines environment", Computers & Operations Research, 2020 Publication	<1%
7	Rafael Perez. "Why is problem-dependent and high-level representation scheme better in a genetic algorithm?", Proceedings of the 1997	<1%

	ACM symposium on Applied computing - SAC 97 SAC 97, 1997 Publication	
8	Submitted to Tilburg University Student Paper	<1%
9	Submitted to Universiti Malaysia Terengganu UMT Student Paper	<1%
10	Submitted to University of Ulster Student Paper	<1%
11	ijcopi.org Internet Source	<1%
12	www.philadelphia.edu.jo Internet Source	<1%
13	Submitted to Heriot-Watt University Student Paper	<1%
14	petertino.github.io Internet Source	<1%
15	ris.utwente.nl Internet Source	<1%
16	www.journal-aprie.com Internet Source	<1%
17	Submitted to Manchester Metropolitan University Student Paper	<1%

18	Submitted to Taibah University Student Paper	<1%
19	ir.lib.nycu.edu.tw Internet Source	<1%
20	"Experimental Algorithms", Springer Science and Business Media LLC, 2008 Publication	<1%
21	Lecture Notes in Computer Science, 2007. Publication	<1%
22	Teofilo F. Gonzalez. "Handbook of Approximation Algorithms and Metaheuristics, Second Edition - Methodologies and Traditional Applications", CRC Press, 2018 Publication	<1%
23	Submitted to Universiti Kebangsaan Malaysia Student Paper	<1%
24	Fiza Zafar, Alicia Cordero, Sadia Mujtaba, Juan R. Torregrosa. "A Hybrid Steffensen–Genetic Algorithm for Finding Multi-Roots of Nonlinear Equations and Applications to Biomedical Engineering", Algorithms, 2025 Publication	<1%
25	Submitted to Loughborough University Student Paper	<1%

26	Pasquale Avella, Igor Vasil'Ev. "A Computational Study of a Cutting Plane Algorithm for University Course Timetabling", Journal of Scheduling, 2005 Publication	<1%
27	Peiran Meng, Zhuo You, Kaixuan Guo, Chunyang Yu, Lidong Gong, Zhongzhi Yang. "Intelligent Algorithm-guided Parameter Learning for the ABEEM Model", Chemical Research in Chinese Universities, 2025 Publication	<1%
28	R. M. R. Lewis. "Guide to Graph Colouring", Springer Science and Business Media LLC, 2021 Publication	<1%
29	Sadaf Naseem Jat, Shengxiang Yang. "A hybrid genetic algorithm and tabu search approach for post enrolment course timetabling", Journal of Scheduling, 2010 Publication	<1%
30	Hasan Iqbal, Arif Sarwat. "Improved Genetic Algorithm-Based Harmonic Mitigation Control of an Asymmetrical Dual-Source 13-Level Switched-Capacitor Multilevel Inverter", Energies, 2024 Publication	<1%

31	Mirna Yunita, Xiabi Liu, Zhaoyang Hai, Rachmat Muwardi. "Adaptive Meta-Loss Networks: Learning Task-Agnostic Loss Functions via Evolutionary Optimization", Computers, Materials & Continua, 2026 Publication	<1%
32	Agbotiname Lucky Imoize, Oleksandr Kuznetsov, Roman Odarchenko, Sergiy Gnatyuk. "Handbook of Cybersecurity - Challenges and Solutions for Emerging Technologies", CRC Press, 2026 Publication	<1%
33	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	<1%
34	Submitted to Queensland Academy for Science, Maths and Technology Student Paper	<1%
35	Submitted to Vardhaman College of Engineering, Hyderabad Student Paper	<1%
36	Submitted to Liverpool John Moores University Student Paper	<1%
37	Submitted to University of Strathclyde Student Paper	<1%

38	Xiangyu Cheng, Min Zhou, Liping Zhang, Zikai Zhang. "ICOA: An Improved Coati Optimization Algorithm with Multi-Strategy Enhancement for Global Optimization and Engineering Design Problems", Biomimetics, 2026 Publication	<1%
39	dspace.mit.edu Internet Source	<1%
40	essay.utwente.nl Internet Source	<1%
41	Siyu Zhang, Xiaowen Zhang, Yanan Xiao, Huiran Tang, Yujing Zhao, Sirou Wang, Ying Peng. "Optimization of municipal solid waste (MSW) collection routes in Hengyang City, china using an enhanced genetic algorithm based on Amap application programming interface (API)", Waste Management, 2026 Publication	<1%
42	Submitted to Universiti Malaysia Sabah Student Paper	<1%
43	Submitted to University of Lancaster Student Paper	<1%
44	Submitted to University of Pretoria Student Paper	<1%
	Submitted to Cardiff University	

45	Student Paper	<1%
46	Submitted to Monash University Student Paper	<1%
47	Rong Yao, Jinsong Wang, YuHang Wei, Siqi Huang, Yu Zhao. "Data-driven inverse kinematics for a 6-DOF robotic manipulator using an MPGA-optimized BP neural network", Engineering Research Express, 2026 Publication	<1%
48	Ruibin Bai, Edmund K. Burke, Michel Gendreau, Graham Kendall, Barry McCollum. "Memory Length in Hyper-heuristics: An Empirical Study", 2007 IEEE Symposium on Computational Intelligence in Scheduling, 2007 Publication	<1%
49	Tassopoulos, I.X.. "Solving effectively the school timetabling problem using particle swarm optimization", Expert Systems With Applications, 201204 Publication	<1%
50	fiveable.me Internet Source	<1%
51	www.intechopen.com Internet Source	<1%

52	"Advanced Intelligent Computing Theories and Applications. With Aspects of Artificial Intelligence", Springer Nature, 2007 Publication	<1%
53	Submitted to The University of the West of Scotland Student Paper	<1%
54	Submitted to University of Sydney Student Paper	<1%
55	www.tandfonline.com Internet Source	<1%
56	Submitted to Adtalem Global Education Student Paper	<1%
57	Submitted to CITY College, Affiliated Institute of the University of Sheffield Student Paper	<1%
58	Christian Blum. "An Introduction to Metaheuristic Techniques", Parallel Metaheuristics, 08/19/2005 Publication	<1%
59	Jia Yu, Jiazhe Song, Huihui Lan, Yugui Zhang. "Multi-objective optimisation of path and space utilisation in landscape garden green space design", PLOS One, 2025 Publication	<1%
	Submitted to New College of the Humanities	

60	Student Paper	<1%
61	Submitted to Niles North High School Student Paper	<1%
62	Submitted to The Robert Gordon University Student Paper	<1%
63	etd.uum.edu.my Internet Source	<1%
64	Submitted to Özyegin Üniversitesi Student Paper	<1%
65	Burke, E.K.. "A graph-based hyper-heuristic for educational timetabling problems", European Journal of Operational Research, 20070101 Publication	<1%
66	Nergiz A. Ismayilova, Mujgan Sağir, Rafail N. Gasimov. "A multiobjective faculty–course–time slot assignment problem with preferences", Mathematical and Computer Modelling, 2007 Publication	<1%
67	eprints.staffs.ac.uk Internet Source	<1%
68	ijettjournal.org Internet Source	<1%

69	www.dituniversity.edu.in Internet Source	<1%
70	Chao, Y.-H.. "Improving the characterization of the alternative hypothesis via minimum verification error training with applications to speaker verification", Pattern Recognition, 200907 Publication	<1%
71	Masri Ayob, Ghaith Jaradat. "Hybrid Ant Colony systems for course timetabling problems", 2009 2nd Conference on Data Mining and Optimization, 2009 Publication	<1%
72	Takada, Noriko. "Development of an Attitude Dynamics Simulation Applet", California Polytechnic State University, 2022 Publication	<1%
73	Ton Duc Thang University Publication	<1%
74	Zeyu Liu, Chengyu Zhou, Junxiang Li, Chenggang Wang, Pengnian Zhang. "Improved Genetic Algorithm-Based Path Planning for Multi-Vehicle Pickup in Smart Transportation", Smart Cities, 2025 Publication	<1%
75	aeuso.org Internet Source	<1%

76	hdl.handle.net Internet Source	<1%
77	ijrpr.com Internet Source	<1%
78	kampouridis.net Internet Source	<1%
79	www.educative.io Internet Source	<1%
80	A. T. Clementson, C. H. Elphick. "Continuous Timetabling Problems", Journal of the Operational Research Society, 2017 Publication	<1%
81	Coelho, João de Deus Salvador Pinheiro Parreira. "Matheuristic Scheduling of a Hybrid Flow Shop in the Paper Industry: a Decomposition-Based Approach.", Universidade do Porto (Portugal) Publication	<1%
82	Jinzhong Zhang, Anqi Jin, Tan Zhang. "A Hybrid Nonlinear Greater Cane Rat Algorithm with Sine–Cosine Algorithm for Global Optimization and Constrained Engineering Applications", Biomimetics, 2025 Publication	<1%
83	Simon J. Cottrell. "Generation of multiple pharmacophore hypotheses using	<1%

multiobjective optimisation techniques", Journal of Computer-Aided Molecular Design, 11/2004
Publication

84	U Aickelin, E K Burke, J Li. "An estimation of distribution algorithm with intelligent local search for rule-based nurse rostering", Journal of the Operational Research Society, 2017 Publication	<1 %
85	archive.org Internet Source	<1 %
86	jims.atu.ac.ir Internet Source	<1 %
87	pureadmin.qub.ac.uk Internet Source	<1 %
88	research-repository.griffith.edu.au Internet Source	<1 %
89	theses.hal.science Internet Source	<1 %
90	www.coursehero.com Internet Source	<1 %
91	www.ijfans.org Internet Source	<1 %

92	"Computational Methods", Springer Science and Business Media LLC, 2006 Publication	<1 %
93	"Feature Selection for Object Detection", Monographs in Computer Science, 2005 Publication	<1 %
94	Aron Gottesman, Michael Leibrock. "How We Got Here", Wiley, 2017 Publication	<1 %
95	Bhandarkar, S.M.. "An edge detection technique using genetic algorithm-based optimization", Pattern Recognition, 199409 Publication	<1 %
96	Submitted to Brunel University Student Paper	<1 %
97	Dexter Romaguera, Jenie Plender-Nabas, Junrie Matias, Lea Austero. "Development of a Web-based Course Timetabling System based on an Enhanced Genetic Algorithm", Procedia Computer Science, 2024 Publication	<1 %
98	G.V. Berghe. "Fitness evaluation for nurse scheduling problems", Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat No 01TH8546) CEC-01, 2001 Publication	<1 %

99	Mark A. Bradford, Russell Q. Bridge, Stephen J. Foster. "Mechanics of Structures and Materials", CRC Press, 2026 Publication	<1 %
100	Meihong Wang. "Grid Task Scheduling Based on Advanced No Velocity PSO", 2010 International Conference on Internet Technology and Applications, 08/2010 Publication	<1 %
101	Mohamed Jarraya. "Efficient Heuristics for Timetabling Scheduling in Blended Learning", International Journal of Computing and Digital Systems, 2024 Publication	<1 %
102	Moritz Mühlenthaler, Rolf Wanka. "Fairness in academic course timetabling", Annals of Operations Research, 2014 Publication	<1 %
103	Operations Research/Computer Science Interfaces Series, 2016. Publication	<1 %
104	Vatroslav Dino Matijaš. "University Course Timetabling Using ACO: A Case Study on Laboratory Exercises", Lecture Notes in Computer Science, 2010 Publication	<1 %

105	William N. Caballero, Phillip R. Jenkins, Brian J. Lunday. "Adaptive course scheduling for the school of advanced air and space studies", Omega, 2026 Publication	<1 %
106	acikbilim.yok.gov.tr Internet Source	<1 %
107	core.ac.uk Internet Source	<1 %
108	eprints.nottingham.ac.uk Internet Source	<1 %
109	escholarship.mcgill.ca Internet Source	<1 %
110	ipfs.io Internet Source	<1 %
111	profdoc.um.ac.ir Internet Source	<1 %
112	repositorio.unal.edu.co Internet Source	<1 %
113	www.decom.ufop.br Internet Source	<1 %
114	Ahmed Redha Mahlous, Houssam Mahlous. "Student timetabling genetic algorithm accounting for student preferences", PeerJ Computer Science, 2023	<1 %

Publication

115	Akin Ozkan, Aydin Ulucan, Ceren Dirik, Kazim Baris Atici. "University course timetabling with multi-section courses, room stability and lecturer preferences: an application in a business school", Computational Management Science, 2025 Publication	<1 %
116	F. Xhafa, J. Kolodziej, L. Barolli, V. Kolici, R. Miho, M. Takizawa. "Evaluation of Hybridization of GA and TS Algorithms for Independent Batch Scheduling in Computational Grids", 2011 International Conference on P2P, Parallel, Grid, Cloud and Internet Computing, 2011 Publication	<1 %
117	Roberto Asín Achá, Robert Nieuwenhuis. "Curriculum-based course timetabling with SAT and MaxSAT", Annals of Operations Research, 2012 Publication	<1 %
118	Genetic Algorithms in Java Basics, 2015. Publication	<1 %

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		